

# 面向对象程序设计

## 第七讲 模板与 STL

李际军 lijijun@cs.zju.edu.cn



# 第 7 章 模板与 STL

- 模板（**template**）是 **C++** 实现代码重用机制的重要工具，是泛型技术（即与数据类型无关的通用程序设计技术）的基础。
- 模板是 **C++** 中相对较新的 语言机制，它实现了与具体数据类型无关的通用算法程序设计，能够提高软件开发的效率，是程序代码复用的强有力工具。
- 本章主要介绍了函数模板和类模板两类，以及 **STL** 库中的几个常用模板数据类型及其应用。



实现多态的方法是 ( )

- ☐ A 将派生类对象赋给基类对象
- ☐ B 通过基类对象的指针调用派生类中虚函数版本
- ☐ C 通过基类的引用调用派生类虚函数版本
- ☐ D 通过派生类指针调用基类的虚函数版本

提交

# 7.1 模板的概念

## 1、有关模板的几个重要概念

### (1) 模板

- 模板是对具有相同特性的函数或类的再抽象，模板是一种**参数多态性**的工具，可以为**逻辑功能相同而类型不同的程序**提供一种**代码共享的机制**。
- 一个模板并非一个实实在在的函数或类，仅仅是一个函数或类的描述，但它**可以接受数据类型作为其调用参数并生成可用的函数或类**，是参数化的函数和类，是创建函数或类的自动化工具。

### (2) 模板的类型

- 函数模板
- 类模板

# 7.1 模板的概念

## (3) 抽象 ◇ 模板

- 从多个具有相同特性的同类事物推导出对该类事物共同特征的统一描述的过程。

- 变量→类型
- 对象→类
- 类→类模板
- 函数→函数模板



抽象

**动物：** 有机物为食，是能够自主运动或能够活动的有感觉的生物

```
int min(int a, int b){return (a<b)?a:b;}  
float min(float a, float b){return (a<b)?a:b;}  
double min(double a, double b){return (a<b)?a:b;}
```

抽象

**T max(T a,T b) 函数模板**  
**{ return a>b?a:b; }**

## (4) 实例化 ◇ 模板函数

- 实例化是抽象的逆过程：由抽象类型定义具体变量的过程。模板实例化过程是：
  - 用实际数据类型代入模板
  - 每一种不同数据类型的实例化，都将生成一份不同的代码



```
T max(T a,T b)  
{ return a>b?  
a:b; }
```



# 7.1 模板的概念

## (5) 模板的抽象与定义

- 某些程序除了所处理的数据类型之外，程序代码和功能完全相同，但为了实现它们，却不得不编写多个与具体数据类型紧密结合的程序。
- 例如，为了求两个 int、float、double、char 类型数中的最小数，需要编写下列函数：

```
int min(int a, int b){return (a<b)?a:b;}
```

```
float min(float a, float b){return (a<b)?a:b;}
```

```
double min(double a, double b){return (a<b)?a:b;}
```

```
char min(char a, char b){return (a<b)?a:b;}
```

.....

.....

- 有什么办法只写一次代码，却可以处理不同数据类型呢？

# 7.1 模板的概念

## ① C 语言通过 **#define** 定义宏

```
define min(x,y) ((x)<(y) ? (x) : (y))
```

– 特点:

- “自动具备多态特征”
- 不适合表达复杂的逻辑
- 简单的文本替代, 不进行任何语法检查, 不安全。

## ② C++ 方法一: 函数重载

– 缺点:

- 相同代码, 多次重复编写!

## ③ C++ 方法二: 函数模板

```
template <typename T>
```

```
T min(T a,T b){ return (a<b)?a:b; }
```

– 特点: 代码复用的有效机制, 可以根据类型生成相应程序代码。

# 7.1 模板的概念

## ④ 模板、模板函数、模板类和对象之间的关系

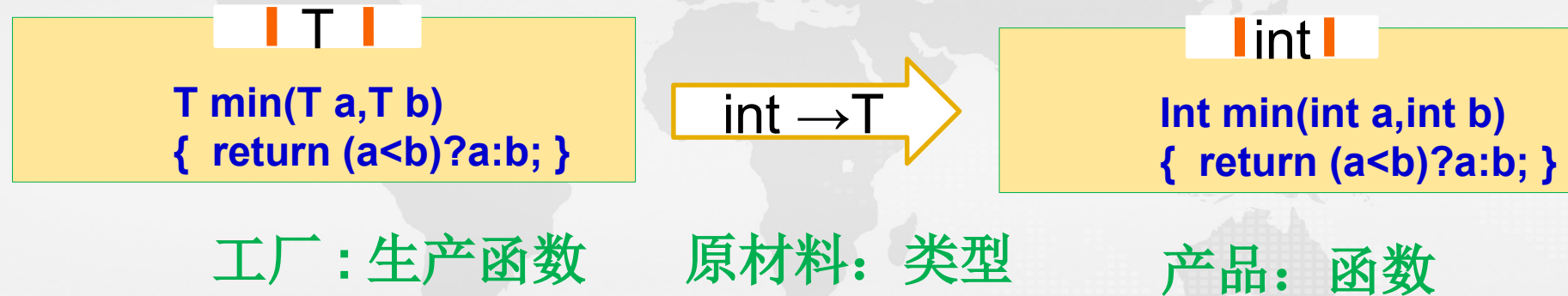




## 7.2 函数模板与模板函数

### 1、函数模板的功能

- 函数模板提供了一种通用的函数行为，该函数行为可以用多种不同的数据类型进行调用，编译器会根据调用类型自动将它实例化为具体数据类型的函数代码，也就是说函数模板代表了一个函数家族。



### 2、函数模板与函数重载的区别

- 与普通函数相比，函数模板中某些函数元素的数据类型是未确定的，这些元素的类型将在使用时被参数化；与重载函数相比，函数模板不需要程序员重复编写函数代码，它可以自动生成许多功能相同但参数和返回值类型不同的函数。

# 7.2.1 函数模板的定义

## 1、函数模板的定义

**template** <typename T1, typename T2,...>

返回类型 函数名 ( 参数表 ) {  
..... // 函数模板定义体  
}

- **template** 是定义模板的关键字；写在一对 <> 中的 T1 ， T2 ， ... 是 **模板参数**，其中的 **typename** 表示其后的参数可以是任意类型。
- **模板参数** 常称为 **类型参数** 或 **类属参数**，在模板实例化（即调用模板函数时）时需要传递的实参是一种数据类型，如 **int** 或 **double** 之类。
- 函数模板的参数表中常常出现模板参数，如 T1 ， T2

# 7.2.1 函数模板的定义

【例 7-1】 求两数最小值的函数模板。

//Eg7-1.cpp

#include <iostream>

using namespace std;

**template <class T>**

**T min(T a,T b) {**

**return (a<b)?a:b;**

**}**

void main(){

    double a=2,b=3.4;

    float c=2.3,d=3.2;

    cout<<"2 , 3 的最小值是: "<<min(2,3)<<endl;

    cout<<"2 , 3.4 的最小值是: "<<min(a,b)<<endl;

    cout<<"'a' , 'b' 的最小值是: "<<min('a','b')<<endl;

    cout<<"2.3 , 3.2 的最小值是: "<<min(c,d)<<endl;

}

程序运行结果如下:

2 , 3 的最小值是: 2

2 , 3.4 的最小值是: 2

'a' , 'b' 的最小值是: a

2.3 , 3.2 的最小值是:

2.3



# 7.2.1 函数模板的定义

## 2、使用函数模板的注意事项

- ① 在定义模板时，不允许 `template` 语句与函数模板定义之间有任何其他语句。

```
template < typename T>
```

```
int x;           // 错误，不允许在此位置有任何语句
```

```
T min(T a,T b){...}
```

- ② 函数模板可以有多个**类型参数**，但每个类型参数都必须用关键字 `class` 或 `typename` 限定。此外，模板参数中还可以出现确定类型参数，称为**非类型参数**。例：

```
template < typename T1, typename T2, typename T3,int T4>
```

```
T1 fx(T1 a, T2 b, T3 c){...}
```

在传递实参时，**非类型参数** `T4` 只能使用常量，如 6

## 7.2.1 函数模板的定义

### ③ 模板类型参数关键字

- `template <typename T1 ,class T2> .`
- 这里的 **class** 和 **typename** 含义相同，与类没有任何关系，它仅表示 T 是一个类型参数，可以是任何数据类型，如 `int`、`float`、`char` 等，或者用户定义的 `struct`、`enum` 或 `class` 等自定义数据类型。建议用 `typename`，以示区别。

### ④ 模板类与普通类文件组织的区别

- 普通函数或类通常把函数或类的声明放在头文件中，把实现代码放在另一个实现文件中，以达到接口与实现分离的目的。
- 模板（包括函数模板和类模板）则不一样，由于在用模板创建（实例化）模板函数或模板类时，编译器必须掌握函数模板或类成员函数模板的确切定义。因此，**必须把模板的声明和定义保存在同一文件中，通常保存在同一头文件中。**

## 7.2.2 函数模板的实例化

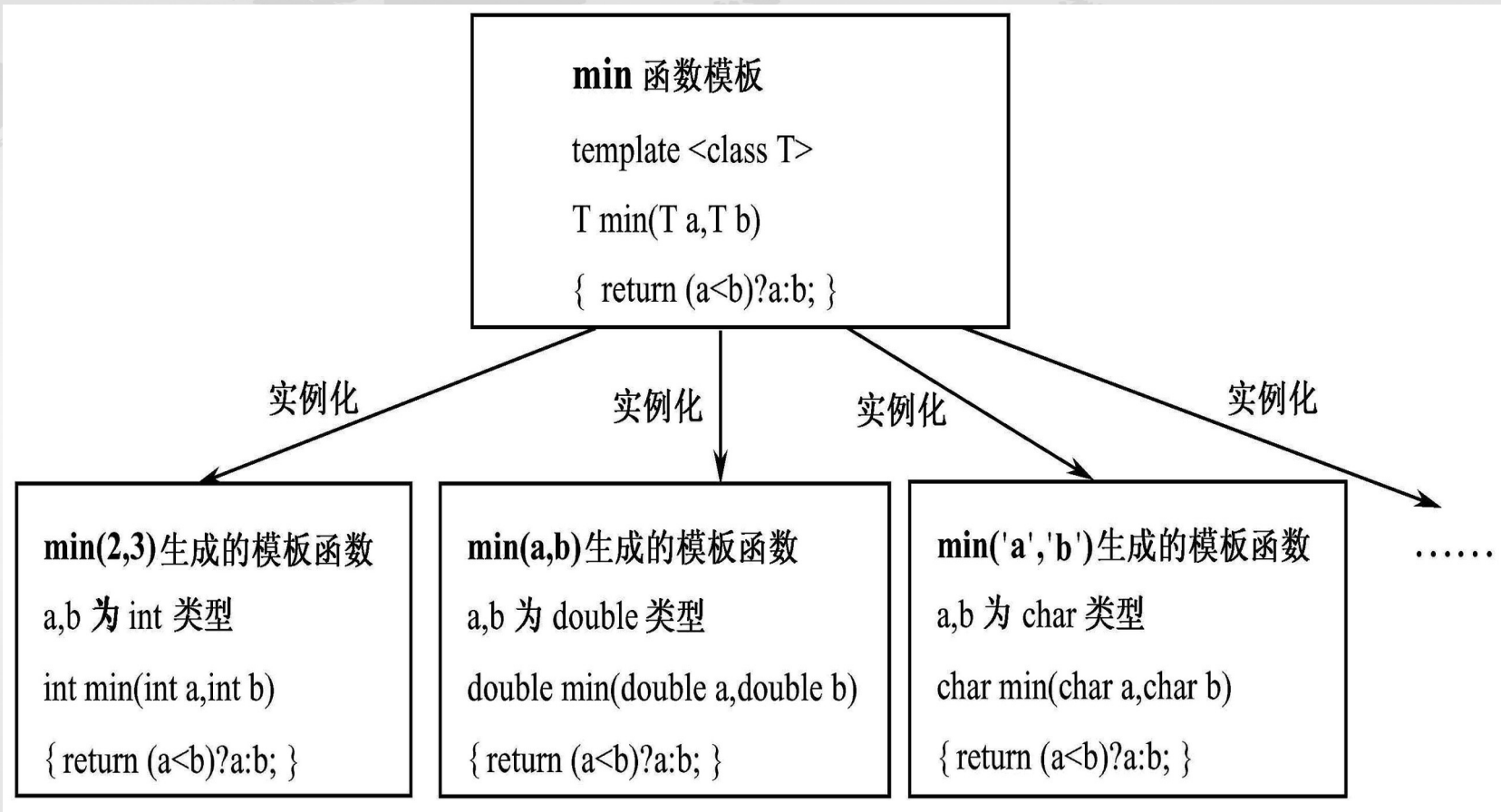
### 1、实例化发生的时机

- 模板实例化发生在调用模板函数时。
- 当编译器遇到程序中对函数模板的调用时，它才会根据调用语句中实参的具体类型，确定模板参数的数据类型，并用此类型替换函数模板中的模板参数，生成能够处理该类型的函数代码，即模板函数。



## 7.2.2 函数模板的实例化

- 函数模板实例化情形



## 7.2.2 函数模板的实例化

2、当多次发生类型相同的参数调用时，**只在第 1 次进行实例化。**

- 对 min 模板的函数调用：

`int x=min(2,3);`

`int y=min(3,9);`

`int z=min(8 , 5);`

```
template <class T>
T min(T a,T b) {
    return (a<b)?a:b;
}
```

↓

```
Int min(int a,int b) {
    return (a<b)?a:b;
}
```

编译器只在第 1 次调用时生成模板函数，当之后遇到相同类型的参数调用时，不再生成其他模板函数，它将调用第 1 次实例化生成的模板函数。

## 7.2.2 函数模板的实例化

### 3、实例化的方式

#### — 隐式实例化

- 编译器能够判断模板参数类型时，自动实例化函数模板为模板函数

```
template <typename T> T max (T, T);
```

```
...
```

```
int i = max (1, 2);
```

```
float f = max (1.0, 2.0);
```

```
char ch = max ('a', 'A');
```

```
...
```

- 隐式实例化，表面上是在调用模板，实际上是调用其实例。



## 7.2.2 函数模板的实例化

### — 显式实例化

- 时机

- 编译器不能判断模板参数类型或常量值
- 需要使用特定数据类型实例化

- 语法形式

- 模板名称 < 数据类型 ,..., 常量值 ,...> ( 参数 )

其中数据类型提供给类型参数，常量值提供给非类型参数

- 示例 1

```
template <class T> T max (T, T);  
...  
int i = max (1, '2'); // error: data type can't be deduced  
int i = max<int> (1, '2');  
...
```

## 7.2.3 模板参数

### 1、模板参数匹配的问题

- C++ 在实例化函数模板的过程中，只是简单地将模板参数替换成调用实参的类型，并以此生成模板函数，**不会进行**参数类型的**任何转换**！

## 7.2.3 模板参数

【例 7-2-01】 求最大值的函数模板。

```
//Eg.cpp
#include <iostream>
using namespace std;
template <class T>
T max(T a,T b) {
    return (a>b)?a:b;
}
void main(){
    double a=2,b=3.4;
    float c=5.1,d=3.2;
    cout<<"2, 3.2 的最大值是: "<<max(2,3.2)<<endl;
    cout<<"a c 的最大值是: "<<max(a,c)<<endl;
    cout<<"'a', 3 的最大值是: "<<max('a',3)<<endl;
}
```



## 7.2.3 模板参数

- 编译本程序，将会产生 3 个编译错误：  
C2782 , “ T max(T,T) ”: 模板参数 “ T ” 不明确  
.....
- 在普通函数的调用过程中， C++ 会对类型不匹配的参数进行隐式的类型转换 。但是，模板实例化过程中不会进行任何形式的参数类型转换，从而导到模板函数的参数类型不匹配，因此产生上述编译错误。

## 7.2.3 模板参数

- 模板参数不匹配的解决方法

( 1 ) 在模板调用时进行参数类型的强制转换

```
cout<<max(double(2),3.2)<<endl;
```

( 2 ) 显式指定函数模板实例化的类型参数

```
cout<<max<double>(2,3.2)<<endl;
```

```
cout<<max<int>('a',3)<<endl;
```

( 3 ) 指定多个模板参数

【例 7-2】 用两个模板参数实现求最大值的函数。

```
#include <iostream>
using namespace std;
template <class T1,class T2>
T1 max(T1 a,T2 b) {
    return (a>b)?a:b;
}
void main(){
    double a=2,b=3.4;
    float c=5.1,d=3.2;
    cout<<"2, 3.2  的最大值是:  "
        <<max(2,3.2)<<endl;
    cout<<"a, c  的最大值是: "<<max(a,c)<<endl;
    cout<<"'a', 3  的最大值是: "<<max('a',3)<<endl;
}
```



## 7.2.2 函数模板的实例化

### 2. 类型与非类型模板参数

- 函数模板参数可以是类属参数，也可以包括普通类型的参数。

【例 7-3】 用函数模板实现数组的选择法排序，

//Eg7-3.cpp

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T>
```

```
void sort(T & a,int n) {  
    for (int i=0;i<n;i++){  
        int p=i;  
        for(int j=i;j<n;j++){  
            if(a[p]<a[j])  
                p=j;  
            int t=a[i];  
            a[i]=a[p];  
            a[p]=t;  
        }  
    }  
}
```

T 是类型参数， n 是非类型参数

```
template <class T>
void display(T& a,int n) {
    for(int i=0;i<n;i++)
    cout<<a[i]<<"\t";
    cout<<endl;
}

void main(){
    int a[]={1,41,2,5,8,21,23};
    char b[]={'a','x','y','e','q','g','o','u'};
    sort(a,7);
    sort(b,8);
    display(a,7);
    display(b,8);
}
```

向模板函数传递非  
类型参数时只能传  
递常量

程序运行结果如下:

```
41 23 21 8 5 2 1
y x u q o g e a
```

## 7.2.3 模板参数

### 3. 模板参数的作用域

- 模板参数遵循普通的作用域范围规则：模板参数的可用范围在其声明之后，直到模板声明或定义结束之前；
- 同局部变量一样，模板参数会隐藏外层作用域中声明的相同名称。但与普通函数不同的是，在模板内不能重用模板参数名。

```
struct A {};
```

```
template<typename A, typename B>
```

```
void f(A a, B b) {
```

```
    A t = a;
```

//t 是 typename A 指定的类型

```
    int B;
```

// 错误，重用模板参数定义变量名称。

```
}
```

- 在 f 模板内，模板参数 A 隐藏了外层作用域定义的结构类型 A。



```
template<typename T>auto f(T a,Tb){.....}
```

针对本函数模板，下面的说法正确的是（ ）

- ☐ A int a=f(3,4.2);
- ☐ B int a=f(4,3),b=f(9,8) 会 2 次实例化函数模板，生成两个 f 函数。
- ☐ C int a=f( "abc" , " 123" );
- ☒ D int a=f<double>f(3,5.4);

# 7.3 类模板

## 7.3.1 类模板的概念

- 类模板可用来设计**结构和成员函数完全相同**，但所处理的**数据类型不同的通用类**。如栈，

双精度栈：

```
class doubleStack{  
    private:  
        double data[size];  
    public:  
        push(double);  
        double pop()  
        double top();  
};
```

字符栈：

```
class charStack{  
    private:  
        char data[size];  
        .....  
};  
    public:  
        push(char);  
        char pop();  
        char top();  
};
```

- 这些栈除了数据类型之外，操作完全相同，就可用类模板实现。

## 7.3.2 类模板的定义

### 1、类模板的声明

```
template<typename T1,typename T2,...>  
class 类名 {  
    .....           // 类成员的声明与定义  
}
```

- 其中 T1 、 T2 是类型参数
- 类模板中可以有多多个模板参数，包括类型参数和非类型参数



## 7.3.2 类模板的定义

- **非类型参数**是指某种具体的数据类型，在调用模板时只能为其提供用相应类型的常数值。

```
template<class T1,class T2,int T3>
```

- T1、T2 是类型参数，T3 是非类型参数。
- 在实例化时，必须为 T1、T2 提供一种数据类型，为 T3 指定一个整常数（如 10），该模板才能被正确地实例化。
- **非类型参数是受限制**
  - 通常可以是整型、枚举型、对象或函数的引用，以及对象、函数或类成员的指针
  - 但不允许用浮点型（或双精度型）、类对象或 void 作为非类型参数。

# 7.3.2 类模板的定义

## 2、类模板的成员函数的定义

方法 1：在类模板外定义，语法

**template** < 模板参数列表 >

返回值类型 类模板名 < 模板参数名称列表 >:: 成员函数名 ( 参数列表 )

{

.....

};

其中

- < 模板参数列表 > 引入的“类型标识符”作为数据类型使用
- < 模板参数名称列表 > 引入的“普通数据类型常量”作为常量使用

方法 2：

- 直接在模板内定义成员函数，与常规成员函数的定义方法相同。

**【例 7-4】** 设计一个堆栈的类模板 **Stack**，在模板中用类型参数 **T** 表示栈中存放的数据，用非类型参数 **MAXSIZE** 代表栈的大小。

```
//Eg7-4.cpp
```

```
//Stack.h
```

```
template<class T,int MAXSIZE>
```

```
class Stack{
```

```
private:
```

```
    T elems[MAXSIZE];
```

```
    int top;
```

```
public:
```

```
    Stack(){top=0;};
```

```
    void push(T e);
```

```
    T pop();
```

```
    bool empty(){return top==0;}
```

```
    bool full(){return top==MAXSIZE;}
```

```
};
```



## 7.3.2 类模板的定义

```
template<class T,int MAXSIZE>
void Stack<T, MAXSIZE>::push(T e) {
    if(top==MAXSIZE){
        cout<<" 栈已满，不能再加入元素了！ ";
        return;
    }
    elems[top++]=e;
}
template<class T,int MAXSIZE>
inline T Stack<T, MAXSIZE>::pop(){
    if(top<=0){
        cout<<" 栈已空，不能再弹出元素了！ "<<endl;
        return 0;
    }
    top--;
    return elems[top];
}
```

< 模板参数列表  
>  
< 模板参数名称列表 >

## 7.3.3 类模板实例化

### 1、类模板实例化的内容

包括**模板实例化**和**成员函数实例化**

### 2、类模板实例化的时间

当用**类模板定义对象**时，引起类模板的实例化

### 3、实例化的方法：

在实例化类模板时，如果模板参数是类型参数，则必须为它指定具体的类型；如果模板参数是非类型参数，则必须为它指定一个常量值。

## 7.3.3 类模板实例化

### 4、实例化案例

如对 **Stack** 类模板，下面的定义将引起实例化

**Stack<int,10> iStack;**

— 编译器实例化 **iStack** 的方法是：

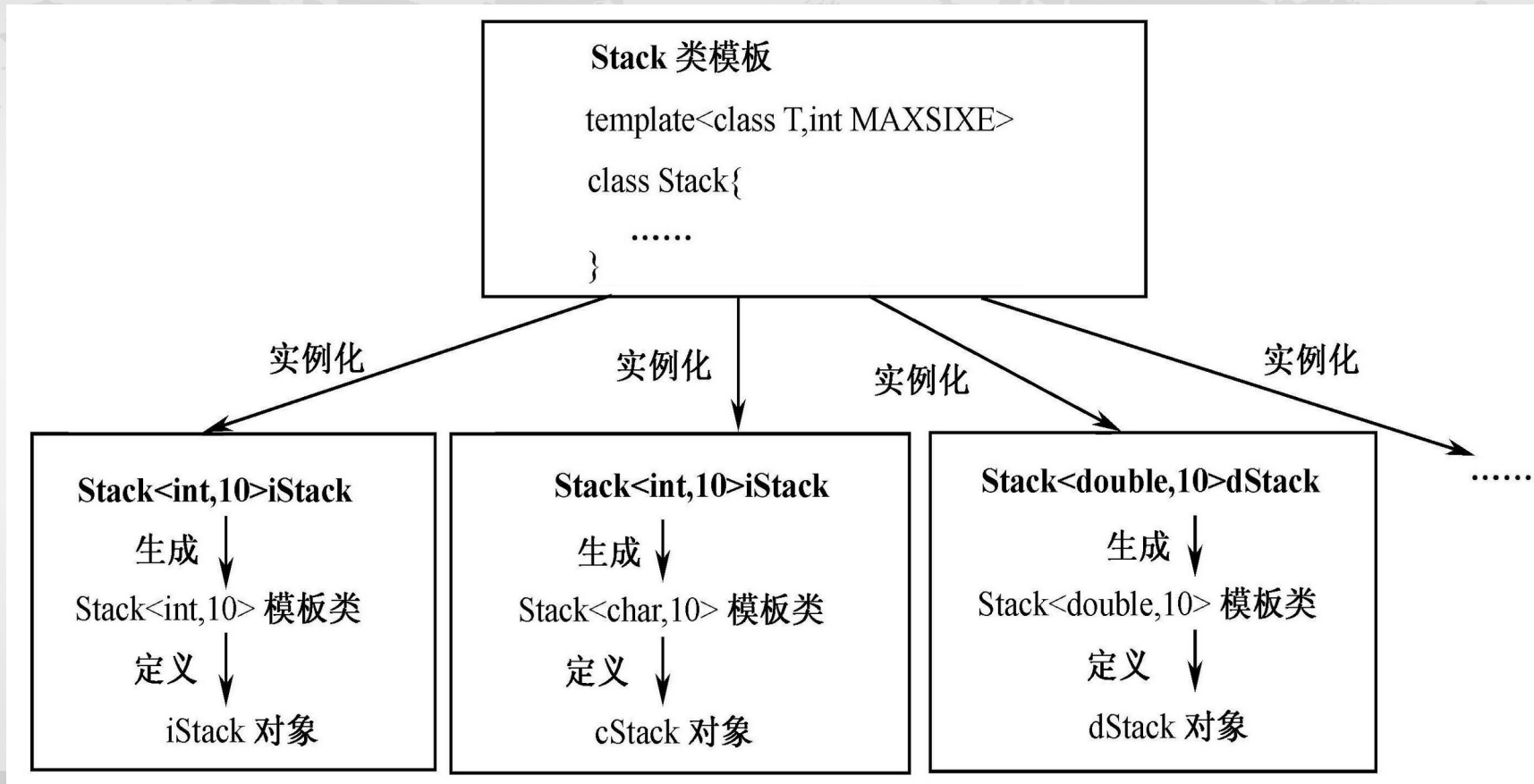
- ① **数据成员实例化**：将 **Stack** 模板声明中的所有数据成员的类型参数 **T** 替换成 **int**，将所有的非类型参数 **MAXSIZE** 替换成 **10**，生成了一个 **int** 类型的模板类。
- ② **成员函数实例化**：
  - 用 **int** 替换无参构造函数中的 **T** 实例化出被调用的构造函数。
  - **未被调用的成员函数不被实例化**（即不生成相应的成员函数）

```
class Stack{
private:
    int elems[10];
    int top;                // 栈顶指针
public:
    Stack(){top=0;};
    void push(int e);       // 入栈操作
    int pop();              // 出栈操作
    bool empty(){return top==0;}
    bool full();}
};
```



## 7.3.3 类模板实例化

**Stack** 模板能够实例化出无穷多的模板类



## 7.3.4 类模板的使用

- 为了使用类模板对象，**显式指定模板实参**进行模板实例化。

```
//Eg7-4b.cpp
#include"stack.h"      // 该头文件的内容见例 7-4 所示的程序清单
#include<iostream>
using std::cout;      // 只使用 std 域名空间中的 cout
using std::endl;      // 只使用 std 域名空间中的 endl
void main(){
    Stack<int,10> iStack;
    Stack<char,10> cStack;
    cout<<"-----intStack----\n";
    int i;
    for(i=1;i<10;i++) iStack.push(i);
    for(i=1;i<10;i++) cout<<iStack.pop()<<"\t";
    cout<<"\n\n-----charStack----\n";
    cStack.push('A'); cStack.push('B');
    cStack.push('C'); cStack.push('D');
    cStack.push('E');
    for(i=1;i<6;i++) cout<<cStack.pop()<<"\t";
    cout<<endl;
}
```

## 7.3.4 类模板的使用

【例 7-5】 对例 7-4 建立的 `Stack` 类模板编写一个函数 `display`，该函数能够读取并显示 `Stack` 模板类建立的栈中的所有元素。

```
//Eg7-5.cpp
#include "stack.h"
#include <iostream>
using namespace std;
template<class T>
void display(Stack<T,10> &s) {
    while( !s.empty())
        cout<<s.pop()<<"\t";
    cout<<endl;
}
void main(){
    Stack<int,10> iStack;
    cout<<"-----intStack----\n";
    for(int i=1;i<10;i++)
        iStack.push(i);
    display(iStack);
}
```



# 7.4 模板设计中的独特问题

## 7.4.1 模板参数类型推导

- 模板参数常用值类型、左值引用、右值引用等。

### 1. 传值模板参数

值类型模板参数的形式如下：

```
template<typename param>  
r_type fun(param p, ...)  
{...}
```

- 调用 `fun()` 时，编译器根据传递给 `p` 的实参类型推导出模板参数 `param` 的实际类型，并以此实例化生成模板函数。
- 编译器在推导模板参数 `param` 类型的过程中，会去除所有加在实参上的类型限定符（包括 `const`、`&` 和 `&&`）以体现函数的值形参语义。因为这样才能够保证模板函数值形参可复制的语义（函数调用时将实参的值复制到值形参变量中）。

# 7.4.1 模板参数类型推导

【例 7-6】 模板传值形参的类型检测。

// Eg7-6.cpp

```
#include<iostream>
```

```
#include<memory>
```

```
#include<typeinfo>
```

```
using namespace std;
```

```
template<typename T>
```

```
void f(T p) {
```

```
    cout<<typeid(p).name()<<endl;
```

```
}
```

```
int main() {  
    int i1 = 0;  
    int& i2 = i1;  
    const int& i3 = i1+8;  
    int&& i4 = 5 + 7;  
    int* i5 = new int(1);  
    unique_ptr<int> i6 {new int};  
    f(2);  
    f(i1);  
    f(i2);  
    f(i3);  
    f(i4);  
    f(i5);  
    // f(i6);    // unique_ptr 不能够被复制  
    f(move(i6));  
}
```

参数类型

int // 从

2

int //

从 i1

int //

从 i2

int //

从 i3

int //

从 i4

int \* \_\_ptr64 //

从 i5

class

std::unique\_ptr<int,s  
truct

std::default\_delete<i  
nt>

# 7.4.1 模板参数类型推导

## 2. 左值引用模板参数

模板参数的形式如下：

```
template<typename param>
```

```
    r_type fun(param &p, ...)
```

```
{...}
```

- 当类型参数 **Param** 为可变引用（左值引用）时，可以向它传递能够推断出地址的实参，如果是 **const** 类型的实参，将保留它的 **const** 限定。
- 但是，不能够接受字面常量和临时变量，这是因为引用参数是可以修改实参值的，但将常量和临时变量传给引用参数，就与该语义相违背了。
- 修改例 7.6，将 **f()** 的类型参数改为引用参数。



## 7.4.1 模板参数类型推导

【例 7-7】 模板传左值形参的类型检测。

// Eg7-7.cpp

```
#include<iostream>
```

```
#include<memory>
```

```
#include<typeinfo>
```

```
using namespace std;
```

```
template<typename T>
```

```
void f(T &p) {
```

```
    cout<<typeid(p).name()<<endl;
```

```
}
```

推出形参 p 的类型

- 当类型参数为左值引用时，无论实参是左值还是右值，推断出的模板参数都是左值。
- 引用当类型参数是右值引用时，只有当实参也是右值时，推断出的模板参数才是右值引用

```
Int main(){
    int i1 = 0;
    int& i2 = i1;
    const int& i3 = i1+8;
    int&& i4 = 5 + 7;
    int* i5 = new int(1);
    unique_ptr<int> i6 {new int};
int main() {
    // f(2); // 错误,
    f(i1); // p 为 int&
    f(i2); // p 为 int &
    f(i3); // p 为 const int&
    f(i4); // p 为 int& , 引用折叠: 参考例 7-8 ,
    f(i5); // p 为 int *&
    f(i6); // p 为 unique_ptr<int>*p
    // f(move(i6)); // unique_ptr 不能够被复制
}
```

# 7.4.1 模板参数类型推导

## 3. 右值引用模板参数

模板参数的形式如下：

```
template<typename param>
```

```
    r_type fun(param && p, ...)
```

```
{...}
```

- 形如 **T&&** 这样的**右值引用**参数，既可以接受左值引用实参，也可以接受右值引用实参，通常被称为**转发引用**。
- 修改例 7.7，将 f( ) 的类型参数改为右值引用参数。

## 7.4.1 模板参数类型推导

【例 7-8】 模板传右值形参的类型检测。

// Eg7-8.cpp

```
#include<iostream>
```

```
#include<memory>
```

```
#include<typeinfo>
```

```
using namespace std;
```

```
template<typename T>
```

```
void f(T &&p) {
```

```
    cout<<typeid(p).name()<<endl;
```

```
}
```

表 7-1 引用折叠规则

| 形 参 | 实 参 |     |
|-----|-----|-----|
|     | &   | &&  |
| T&  | T&  | T&  |
| T&& | T&  | T&& |

推出形参 p 的类型

- 当类型参数为左值引用时，无论实参是左值还是右值，推断出的模板参数都是左值。
- 引用当类型参数是右值引用时，只有当实参也是右值时，推断出的模板参数才是右值引用

```
Int main(){
    int i1 = 0;
    int& i2 = i1;
    const int& i3 = i1+8;
    int&& i4 = 5 + 7;
    int* i5 = new int(1);
    unique_ptr<int> i6 {new int};
    int main() {
        // f(2);
        f(i1);
        f(i2);
        f(i3);
        f(i4);
        f(i5);
        f(i6);
        // f(move(i6));
    }
    // 错误,
    // p 为 int&
    // p 为 int &
    // p 为 const int&
    // p 为 const int&
    // p 为 int *&
    // p 为 unique_ptr<int>*p
    // unique_ptr 不能够被复制
```



# 7.4 模板设计中的独特问题

## 7.4.2 内联与常量函数模板

- 函数模板和类模板都可以定义为 inline 和 constexpr 函数。方法是将 inline 和 constexpr 放在模板参数列表之后，函数返回类型之前。

- template <class T>

inline T min(T a, T b) { return (a<b) ? a : b; }

- template <class T>

constexpr T min(T a, T b) { return (a<b) ? a : b; } 11C<sub>++</sub>

- 下面模板声明则是错误的，inline 关键字的位置不对。

inline template <class T>

T min(T a, T b) { return (a<b) ? a : b; }

## 7.4.3 默认模板实参

11C++

- 模板参数可以指定默认值（包括函数模板和类模板）
- 遵守与函数默认值同样的规则：一旦为某个模板参数指定了默认值，则它右边的模板参数都应该有默认值
- **【例 7-9】** 设计比较两个不同类型数字大小的函数模板 `compare`，第二个模板参数的类型默认为 `double`。当第 1 个参数大于第 2 个参数时返回 1，大于第 2 个参数时返回 -1，相等时返回 0。

## 7.4.2 默认模板实参

11C<sub>++</sub>

```
//Eg7-9.cpp
#include<iostream>
using namespace std;
template <typename T, typename D = double> // 默认模板参数
int compare(T t = 0, D u = 0) {
    if (t > u) return 1;
    else if (t < u) return -1;
    else return 0;
}
void main() {
    cout << compare(10, 'a') << "\t";           //compare<int,char>(10,'a')
    cout << compare<int, char>() << "\t";       //compare<int,char>(0,0)
    // 下面两次 compare 调用都使用了默认模板参数 double
    cout << compare(20) << "\t";               //compare<int,double>(20,0),
    cout << compare<int>() << endl;           //compare<int,double>(0,0)
    //compare();                               // 错误：不能确定模板参数 T 的类型
}
```

程序运行的结果是：

-1 0 1 0



下面关于函数默认值正确的是 ( )

- ☐ A `int f(int a=1,int b,int c);`
- ☐ B `template<class T = int, class T2>  
T f1(T a = 0, T2 j) {}`
- ☒ C `template<class T = int, class T2=char>  
T f1(T a = 0, T2 j= '0' ) {}`
- ☐ D `int g(int a=0, int k);`

提交

# 7.4.3 仿函数应用

## 1. 仿函数基本概念

- 仿函数又叫函数对象，可以像调用函数一样使用对象。实际是类的函数调用运算符函数 `operator()` 的重载函数。定义形式：

```
class A {  
    A operator()( 参数表 ) {...}           // 仿函数  
}
```

- 与普通函数区别：仿函数可以灵活应用于自身或其他函数。例如：

```
A obj;  
obj(...);           // 调用仿函数，参数表与 operator() 的参数表相匹配
```

【例 7-10】 设计一个通用求和、积的函数模板 `accumulate()`，调用类模板 `Add` 和 `Mul` 的仿函数计算数据区间的和或积。

```
#include<iostream>
#include<vector>
using namespace std;
template<typename T>
class Add {
public:
    T operator()(const T& x, const T& y) { return x + y; }
};
template<typename T>
class Mul {
public:
    T operator()(const T& x, const T& y) { return x * y; }
};
template<typename iter,typename T, typename function>
T accumulate(iter begin, iter end, T init, function f) { //f 是函数对象，接收函数名为参数
    for(iter it = begin; it != end; ++it)
        init = f(init, *it);
    return init;
}
int main() {
    vector v1 = {2.1, 3.1, 4.1, 5.1, 7.1};
    vector<double> v2 = {2.0, 3.0, 4.0, 5.0, 7.0};
    double sum = accumulate(v1.begin(), v1.end(), 0.0, Add<int>{});
    double mul = accumulate(v2.begin(), v2.end(), 1.0, Mul<double>{});
    cout<<"sum = "<<sum<<"\tmul = "<<mul<<endl;
}}
```

程序运行结果如下：  
sum = 21 mul = 840



## 7.4.5 成员模板

### 1、成员模板的概念

- 可以把类（普通类和类模板）的某些成员函数设置为模板，称为成员模板。

### 2、成员模板使用规则

- 成员模板的定义方法与普通函数模板相同；
- 但是，成员模板是类的成员，可以访问类的所有成员，使用与类成员访问权限和作用域限定的相同规则。
- 成员模板不能是虚函数。

**【例 7-11】** OutArray 是一个数组输出的代理类，为它设计一个成员模板，用于输出指定大小的不同类型数组值。

- 问题分析

- 重载 **OutArray** 类的函数调用运算符函数 **operator()** 为成员模板，接收模板数组参数，并输出该数组中的数据元素。

```
//Eg7-11.cpp
```

```
#include<iostream>
```

```
using namespace std;
```

```
class OutArray {
```

```
public:
```

```
    OutArray(ostream& o = cout) :os(o) {}
```

```
    template<typename T> void operator()(T *a, int n) {
```

```
        for (int i = 0; i < n; i++)
```

```
            os << a[i] << "\\t";
```

```
        os << endl;
```

```
    }
```

```
private:
```

```
    ostream &os;
```

```
};
```

## 7.4.5 成员模板

```
int main() {  
    double d[] = { 1.2,3.4,5.6,8,9,21 };  
    const char *c[] = { "abc","efg","der","aa" };  
    OutArray out;           // 定义 OutArray 类对象  
    out(d,6);               // 实例化 OutArray::operator(double *,int)  
    out(c,4);               // 实例化 OutArray::operator(double *,int)  
    return 0;  
}
```

程序运行结果如下：

|     |     |     |    |   |    |
|-----|-----|-----|----|---|----|
| 1.2 | 3.4 | 5.6 | 8  | 9 | 21 |
| abc | efg | der | aa |   |    |



## 7.4.5 可变参数函数模板 11C<sub>++</sub>

### 1、关于可变参数函数模板

- 参数类型和个数都不确定的函数模板，即可变参数模板。
- 此模板为功能需求明确，但数据类型和参数个数不确定的程序设计提供了更大的灵活性。

### 2、可变参数函数模板的定义方法

```
template<typename T1, typename... T2>  
r_type f(T1 p, T2... arg)  
{  
    .....  
}
```

- 其中，T2 就是可变模板参数，称为参数包，可以是零个或多个类型不同的模板参数。

## 7.4.5 可变参数函数模板 11C<sub>++</sub>

【例 7-12】设计 max 函数模板，能够从任意多个数字中计算出最大值。

```
//Eg7-12.cpp
```

```
#include<iostream>
```

```
using namespace std;
```

```
template<typename T mymax(T t)
```

```
{
```

```
    return t;
```

```
}
```

// 结束

可变参数模板的运行过程和递归程序相似。需要定义结束模板递归结束的函数模板！

```
template<typename T1, typename... T2> double max(T1 p, T2... arg)
```

```
{
```

```
    double ret = mymax(arg...);
```

// 包扩展

```
    if (p > ret) return p;
```

```
    else return ret;
```

```
}
```

## 7.4.5 可变参数函数模板 11C<sub>++</sub>

```
int main(){
    cout << mymax(1, 12, 3, 4, 20) << "\t";    // 输出: 20
    cout << mymax('5', 32, '2', 23.0) << "\t";    // 输出: 53 ('5' 的 ASCII)
    cout << mymax('a', 'z', 2) << "\t";          // 输出: 122 ('z' 的 ASCII)
    cout << mymax(2, 3.2) << endl;                // 输出: 3.2
    return 0;
}
```

### 3、可变参数函数模板的执行过程

#### (1) 包扩展

可变参数函数通常都是递归调用的：函数执行时先处理包中的第一个实参，完成后调用包中的剩余实参调用函数，称为**包扩展**。类似于下面的程序结构：



## 7.4.5 可变参数函数模板 11C<sub>++</sub>

- 可变参数函数模板处理过程

```
return_type f(T1 p, T2... arg)
```

```
{
```

将 **arg** 中的第 1 个参数取出给 **p** ;  
f(...arg 中除第 1 个参数  
之外的其余参数) // 递归

.....

```
}
```

```
template<class T1, class... T2>  
double mymax(T1 p, T2... arg)  
{  
    double ret = mymax(arg...);  
  
    if (p > ret) return p;  
    else      return ret;  
}
```

- 例如, “max('a', 'z', 2)”调用的包扩展过程如下, 分 4 次调用才结束。

① mymax('a', 'z', 2);

// 包扩展函数

p='a' ret=('z',2)

② mymax('z', 2)

// 包扩展函数

p='z' ret=(2)

③ mymax(2)

// 值为 2, 结束递归调用

# mymax('a', 'z', 2) 的调用过程

```
template<class T1, class... T2>
double mymax(T1 p, T2... arg)
{
    double ret = mymax(arg...);

    if (p > ret) return p;
    else      return ret;
}
```

**mymax('a', 'z', 2)**

mymax('a', 'z', 2);

P='a';

ret= mymax('z', 2);

P='z';

ret =mymax( 2);

return 122;

ret = 122;

ret = 2;

## 7.4.6 可变参数函数模板 11C<sub>++</sub>

- (2) 包扩展结束

- 为了终止递归，通常会为可变参数模板定义一个非可变参数的普通函数。
- 比如，例 7-12 中的 “double mymax()” 就是用于结束递归的函数，它会在变参模板函数的参数处理完毕后被调用到，用于结束 mymax 函数的递归调用。



## 7.4.7 元编程的基本概念 11C<sub>++</sub>

- 元编程由来
  - 1994 年，Erwin Unruh 写了一个求解小于 N 的全部素数的 C++ 程序，该程序并不能通过编译，但编译器在错误信息中显示出了求解结果。这个程序证明了编译器具有计算能力，程序的功能并非必须在运行期才能够实现，在编译期也是可以实现的。
- 基本概念
  - 元编程（meta-programming）正是基于编译器具有计算能力这个基础演化出来的一种编程技术，其思想是：利用模板技术，编写出能够在编译期间计算出运行时需要的常数、类型、代码的元模板，该模板能够读取、分析、转换或者生成其他程序，甚至在运行时修改程序自身（反射），有人形象地称之为“**编程序的程序**”

## 7.4.7 元编程的基本概念

11C<sub>++</sub>

【例 7-13】 设计可变参数的函数模板 `mysum( )` 计算任意多个类型不确定数字的总和

```
#include<iostream>
template<typename T>
T mysum(T s) { return s; }
```

```
template<typename T, typename ... P>
T mysum(T t, P ... x) { // C++ 14 , 可用 auto mysum(T t, P ... x)
    return t + mysum(x ...);
}
```

```
int main() {
    auto s1 = mysum(2.0f, 4.3f, 10.1f, -3.6f);
    auto s2 = mysum(2, 4.3f, 10u, -45.2);
    std::cout<<"s1 = "<<s1<<std::endl;
    std::cout<<"s2 = "<<s2<<std::endl;
}
```

程序结果：

```
s1 = 12.8
s2      =      -
2147483648
```

程序运行结果是错误的，根

元编程

译期间完成类型计算！

## 【例 7-14】 基于元模板的可变参数函数模板计算多类型数字总和的函数 mysum( )。

```
#include<iostream>
using namespace std;
template<typename ...P> struct SumType;
template<typename T> struct SumType<T>
{ using type = T; };
template<typename T, typename ...P>
struct SumType<T,P...> {
    using type = decltype(T() + typename SumType<P...>::type());
};
template<typename ...P>
using sumRetType = typename SumType<P...>::type;

template<typename T>
T mysum(T s) { return s; }
template<typename T, typename ...P>
sumRetType<T, P ...> mysum(T t, P ...x)
{ return t + mysum(x ...); }

int main() {
    auto s1 = mysum(2.0f, 4.3f, 10.1f, -3.6f);
    auto s2 = mysum(2, 4.3f, 10u, -45.2);
    std::cout<<"s1 = "<<s1<<std::endl;
    std::cout<<"s2 = "<<s2<<std::endl;
}
```

定义 mysum 的返回值类型 SumType

为编译器设计计算类型 SumType 的递归算法

变参的返回类型为：SumType。  
编译器会根据调用实参的类型在编译期利用上面的算法计算出 SumType 的实际类型，然后再实例化出具体数据类型的 mysum 函数

程序结果：

s1 = 12.8

s2 = -28.9



## 【例 7-15】 运用 C++ 14 标准的 `common_type_t` 通用数据类型，设计泛型可变参数求和函数模板。

```
// Eg7-15.cpp
#include<iostream>
#include<type_traits>
using namespace std;
```

此头文件定义通用模板参数类型： `common_type`

```
template<typename T>
T mysum(T s) { return s; }

template<typename T, typename ... P>
common_type_t<T, P ... > mysum(T t, P ... x) {
    return t + mysum(x ...);
}
```

```
int main() {
    auto s1 = mysum(2.0f, 4.3f, 10.1f, -3.6f);
    auto s2 = mysum(2, 4.3f, 10u, -45.2);
    cout<<"s1 = "<<s1<<endl;
    cout<<"s2 = "<<s2<<endl;
}
```

// C++ 14

在 C++ 11 的标准头文件 `<type_traits>` 中提供了类似于例 7-14 中 `SumType` 的通用类型 `comm_type`（在 C++ 14 及以上标准中的名称为 `comm_type_t`）

运用 `comm_type_t` 和可变参数函数模板，改写后的 `mysum()` 版本。两者具有完全相同的功能。

程序结果：

`s1 = 12.8`

`s2 = -28.9`

## 7.4.8 模板重载、特化、非模板函数及调用次序

# 1. 模板重载

- 模板可以被另一个模板或普通函数重载
- 同函数重载的规则相同，要求重载的同名函数模板必须具有不同的形参表。

【例 7-16】设计从两个数中找出最大值的函数模板，并重载该函数模板，实现从任意三个数中找出最大值的函数模板。

```
//Eg7-16.cpp  
#include<iostream>  
#include<string>  
using namespace std;
```

```
template <typename T>
inline T const& max(T const& a, T const& b){
    return a < b ? b : a;
}
template <typename T>
inline T const& max(T const& a, T const& b, T const& c){
    return max(max(a, b), c);
}
int main(){
    int a = 5, b = 12;
    string s1 = "aString1", s2 = "aZtring2";
    const char* c1 = "hellow template override!";
    const char* c2 = "hellow C++ 11!";
    const char* c3 = "hellow everyone!";
    cout << max(7, 42, 32) << endl;    //L1
    cout << max(a, b) << endl;        //L2
    cout << max(s1, s2) << endl;      //L3
    cout << max(c1, c2, c3) << endl;  //L4
    cout << max(c1, c3) << endl;     //L5
}
```

程序运行结果如下:

42 //L1 的输出

12 //L2 的输出

aZtring2 //L3 的输出

hellow everyone! //L4 的输出, 错误

hellow everyone! //L5 的输出, 错误

**L4、L5 的错误原因是重载的 max 函数模板不能从 char \* 类型的字符串中找出正确的最大值。**

**解决方法是特化模板或定义正确的变通函数!**



## 7.4.8 模板重载、特化、非模板函数及调用次序

### 2. 模板特化

#### (1) 模板特化的原因

- 模板虽然能够实例化出实用于各种数据类型的可用函数或类，但要**让一个模板实现对全部数据类型的正确处理，却不一定做得到**。比如，在例 7-9 中，`max` 模板就不能够正确计算字符串的最大值。因为，`char*` 类型的字符串需要用 `strcmp` 函数而不是 “`<`” 运算符比较其大小。
- 为了解决这一问题，C++ 允许为模板定义针对某种数据类型的替代版本，称为模板的特化。

#### (2) 模板特化的语法形式

**template <>**

**用具体类型替换模板参数的函数模板或类模板**

- `<>` 中不需要任何内容，表示模板特化。

#### (3) 特化模板的设计方法

- 特化模板必须与原模板具有相同的结构，在设计某种数据类型的特化模板时，**只需要把原模板中的类型参数替换成指定数据类型后，重新编写函数模板（或类模板成员函数）的实现代码就行了。**

## 7.4.5 模板重载、特化、非模板函数及调用次序

【例 7-17】为例 7-6 的 `max` 函数模板提供特化版，找出 2 个 `char*` 类型字符串中的最大值。

- 解决模板不能正确进行字符串大小比较的问题
  - 例 7-16 的模板不能正确正字符串的大小比较，解决方法是在原程序中添加处理 `const char*` 最大值计算的 `max` 特化模板函数。
  - 方法是：用 `const char*` 替换 `max` 模板中的 `T`，其余代码不作任何修改。

## 7.4.8 模板重载、特化、非模板函数及调用次序

- 不修改程序的任何代码（包括 `main` 函数），只在原来的程序中添加下面的特化模板

//Eg7-17

.....

template<>

const char\* const& max(const char\* const& a, const char\* const& b)

{

return strcmp(a, b) < 0 ? b :

}

```
template <typename T>
inline T const& max(T const& a, T const& b){
    return a < b ? b : a;
}
```

程序运行结果如下，这次是正确的了！最后两行是特化模板输出的。

42

12

aZtring2

hellow template override!

// 正确

hellow template override!

// 正确



## 7.4.8 模板重载、特化、非模板函数及调用次序

### 3. 为模板补充普通函数

- 针对模板不能正确处理的特定数据类型，除了提供处理该类型的特化版本之外，还可以**提供处理该类型的普通函数版本**。

**【例 7-18】**在例 7-17 中添加从两个 `const char*` 类型的字符串中找出最大值的普通 `max` 函数。

- 在例 7-17 中添加普通 `max` 函数，其余代码不作任何修改，完整的程序代码如下：

```
//Eg7-18.cpp
```

```
#include<iostream>
```

```
#include<string>
```

```
using namespace std;
```

```
template <typename T>
```

```
inline T const& max(T const& a, T const& b){
```

```
    cout<<"3:\t";    return  a < b ? b : a;
```

```
}
```

```
template <typename T>
```

```
inline T const& max(T const& a, T const& b, T const& c){
```

```
    return max(max(a, b), c);
```

```
}
```

```
template<>
```

```
// 特化
```

```
const char* const& max(const char* const& a, const char* const& b){
```

```
    cout<<"2:\t";    return  strcmp(a, b) < 0 ? b : a;
```

```
}
```

## 7.4.8 模板重载、特化、非模板函数及调用次序

```
inline char const* max(char const* a, char const* b){  
    cout<<"1:\t"; return std::strcmp(a, b) < 0 ? b : a;  
}
```

```
int main(){  
    int a = 5, b = 12;  
    string s1 = "aString1", s2 = "aZtring2";  
    const char* c1 = "hellow template override!";  
    const char* c2 = "hellow C++ 11!";  
    const char* c3 = "hellow everyone!";  
    cout << max(7, 42, 32) << endl; //L1 , 输出: 42  
    cout << max(a, b) << endl; //L2 , 输出: 12  
    cout << max(s1, s2) << endl; //L3 , 输出: aZtring2  
    cout << max(c1, c2, c3) << endl; //L4 , 输出: hellow template  
    override!  
    cout << max(c1, c3) << endl; //L5 , 输出: hellow template override!  
}
```

```
3: 3: 42  
3: 12  
3: aZtring2  
2: 2: hellow template override!  
1: hellow template override!
```



## 7.4.8 模板重载、特化、非模板函数及调用次序

# 4. 函数参数匹配与调用次序

当一个程序同时拥有重载模板、特化模板和普通重载函数，编译器的选择次序如下：

（1）在众多符合调用条件的函数中**选择最佳匹配**的函数，在匹配非模板函数时会进行参数类型转换。

（2）如果模板函数、模板特化函数和普通函数都符合函数调用要求，按以下次序调用：

- ① **优先调用普通函数**；
- ② 如果没有普通函数，优先**调用模板特化函数**；
- ③ 当没有普通函数和特化模板函数时**才调用模板函数**。如果有多个重载模板函数都符合要求，就选择精确匹配的模板函数；
- ④ 如果多个模板函数都差不多，就**产生二义性错误**。

当程序中同时有模板函数、模板特化函数、普通函数时，其调用次序是（ ）。

- ☐ A 模板函数→特化函数→普通函数
- ☐ B 特化函数→模板函数→普通函数
- ☒ C 普通函数→特化函数→模板函数
- ☐ D 模板函数→特化函数→普通函数

# 7.5 STL





# 7.5.1 函数对象

## 1、函数对象的概念

- STL 标准库为每个算术运算和关系运算定义了一个对应的运算符模板类，能够实现对应的运算符操作，称为**函数对象**。
- 函数对象的定义在 **functional** 头文件中，如表所示。

| 算术对象          | 关系对象             | 逻辑运算对象         |
|---------------|------------------|----------------|
| plus<T>       | equal_to<T>      | logical_and<T> |
| minus<T>      | not_equal_to<T>  | logical_or<T>  |
| multiplies<T> | greater<T>       | logical_not<T> |
| divides<T>    | greater_equal<T> |                |
| modulus<T>    | less<T>          |                |
| negate<T>     | less_equal<T>    |                |

## 7.5.1 函数对象

### 2、函数对象的应用

- 在程序中可以用这些模板定义对象，实现相应的运算。

【例 7-19】 函数对象应用示例。

//Eg7-19.cpp

```
#include<iostream>
```

```
#include<functional>
```

```
#include<algorithm>
```

```
using namespace std;
```

```
void main() {  
    int a[] = { 3,1,7,0,-3,2,8,-5 };  
    plus<int> iadd;  
    minus<double> dm;  
    less<int> les;  
    int s = iadd(5, 6);  
    double d = dm(25, 5);  
    cout << "s=" << s << "\td=" << d << "\t";  
    if (les(5, 7)) cout << "5<7"<<endl;  
    else cout << "5>7" << endl;  
    sort(a, a + 8, less<int>());           // 从小到大排序 a 数组  
    for (int i = 0; i < 8; i++) cout << a[i]<<"\t";  
    cout << endl;  
    sort(a, a + 8, greater<int>());       // 从大到小排序 a 数组  
    for (int i = 0; i < 8; i++) cout << a[i]<<"\t";  
    cout << endl;  
}
```

程序运算结果如下:

s=11 d=20 5<7

|    |     |   |   |   |   |    |
|----|-----|---|---|---|---|----|
| -5 | -30 | 1 | 2 | 3 | 7 | 8  |
| 8  | 7   | 3 | 2 | 1 | 0 | -3 |
|    |     |   |   |   |   | -5 |



## 7.5.2 顺序容器

### 1、C++ 容器的概念及类型

- 容器（`container`）是用来存储其他对象的对象，它是用模板技术实现的。
- STL 的容器包括以下三类。

#### ① 顺序容器

向量（`vector`）、链表（`list`）、双端队列（`deque`）。

#### ② 关联容器

集合（`set`）、多重集合（`multiset`）

#### ③ 容器适配器

堆栈（`stack`）、队列（`queue`）、`priority_queue`（优先队列）

## 表 7-2 STL 中的容器及头文件名

| 容器名            | 头文件名     | 说 明                    |
|----------------|----------|------------------------|
| vector         | <vector> | 向量，从后面快速插入和删除，直接访问任何元素 |
| list           | <list>   | 双向链表                   |
| deque          | <deque>  | 双端队列                   |
| set            | <set>    | 元素不重复的集合               |
| multiset       | <set>    | 元素可重复的集合               |
| stack          | <stack>  | 堆栈，后进先出（ LIFO ）        |
| map            | <map>    | 一个键只对于一个值的映射           |
| multimap       | <map>    | 一个键可对于多个值的映射           |
| queue          | <queue>  | 队列，先进先出（ FIFO ）        |
| priority_queue | <queue>  | 优先级队列                  |

## 表 7-3 所有容器都具有的成员函数

| 成员函数名      | 说 明                                        |
|------------|--------------------------------------------|
| 默认构造函数     | 对容器进行默认初始化的构造函数，常有多，用于提供不同的容器初始化方法         |
| 拷贝构造函数     | 用于将容器初始化为同类型的现有容器的副本                       |
| 析构函数       | 执行容器销毁时的清理工作                               |
| empty()    | 判断容器是否为空，若为空返回 true ， 否则返回 false           |
| max_size() | 返回容器最大容量，即容器能够保存的最多元素个数                    |
| size       | 返回容器中当前元素的个数                               |
| operator=  | 将一个容器赋给另一个同类容器                             |
| operator<  | 如果第 1 个容器小于第 2 个容器，则返回 true ， 否则返回 false   |
| operator<= | 如果第 1 个容器小于等于第 2 个容器，则返回 true ， 否则返回 false |
| operator>  | 如果第 1 个容器大于第 2 个容器，则返回 true ， 否则返回 false   |
| operator>= | 如果第 1 个容器大于等于第 2 个容器，则返回 true ， 否则返回 false |
| swap       | 交换两个容器中的元素                                 |



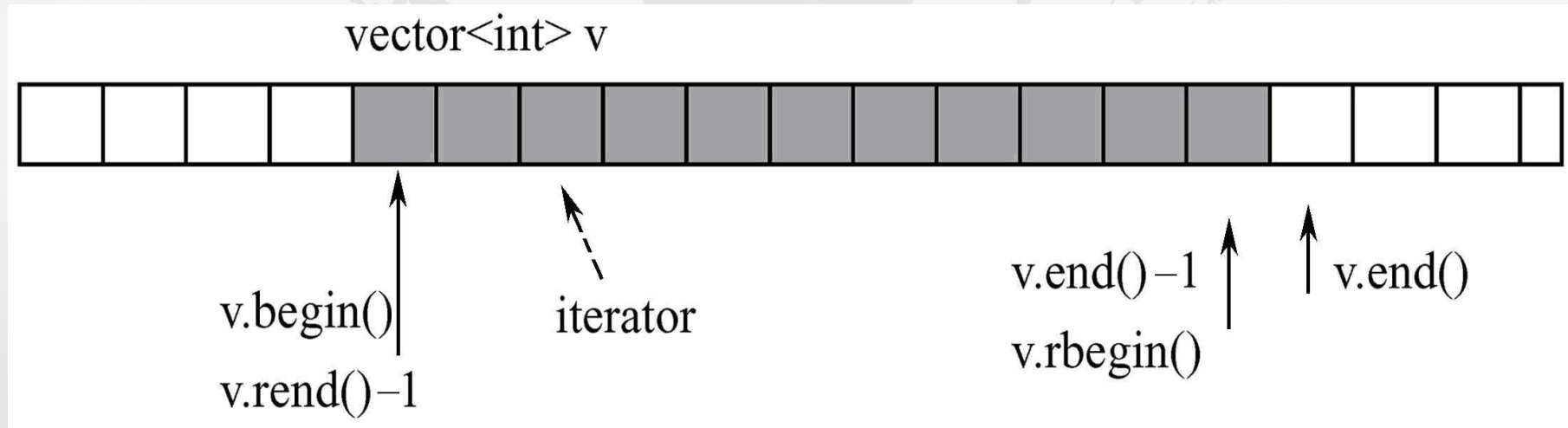
**表 7-4 顺序和关联容器共同支持的成员函数**

| 成员函数名    | 说 明                |
|----------|--------------------|
| begin()  | 指向第一个元素            |
| end()    | 指向最后一个元素的后一个位置     |
| rbegin() | 指向按反顺序的第一个元素的前一个位置 |
| rend()   | 指向按反顺序的末端位置        |
| erase()  | 删除容器中的一个或多个元素      |
| clear()  | 删除容器中的所有元素         |

## 7.5.2 顺序容器

### 1. Vector

- vector** 是向量容器，它具有存储管理的功能，在插入或删除数据时，**vector** 能够自动扩展和压缩其大小。可以像数组一样使用 **vector**，通过运算符 `[]` 访问其元素，但它比数组更灵活，当添加数据时，**vector** 的大小能够自动增加以容纳新的元素。图是向量的一个示意图。



## 7.5.2 顺序容器

### 【例 7-20】 **vector** 向量的应用举例。

```
//Eg7-20.cpp
#include<iostream>
#include<vector>           // 向量头文件
using namespace std;

void display(vector<int> &v) {
    while(!v.empty()){
        cout<<v.back()<<"\t"; // 输出向量的尾部元素
        v.pop_back();           // 删除向量尾部元素
    }
    cout<<endl;
}
```



## 7.5.2 顺序容器

```
void main(){
    int a[]={1,2,3,4,5,6};
    vector<int> v1, v2;           // 定义只有 0 个元素的向量 v1 、 v2
    vector<int> v3(a,a+6);       // 定义向量 v3 ， 并用 a 数组初始化该向量
    vector<int> v4(6);           // 定义具有 6 个元素的向量 v4
    v1.push_back(10);            // 在 v1 向量的尾部加入元素 10
    v1.push_back(11);
    v1.push_back(12);
    v1.insert(v1.begin(),30);    // 将 30 插入到 v1 向量的最前面
    v2=v1;                       // 复制 v1 到 v2
    v3.assign(3,10);
    cout<<"v1: "; display(v1);
    cout<<"v2: "; display(v2);
    cout<<"v3: "; display(v3);
    v4[0]=10; v4[1]=20;          // 用数组初始化 v4
    v4[2]=30; v4[3]=40;
    cout<<"v4: ";
    for(int i=0;i<6;i++)
        cout<<v4[i]<<"\t";
    cout<<endl;
    v4.resize(10);              // 重置向量 v4 的大小， 已有元素不受影响
    cout<<"v4: "; display(v4);
}
```

程序运行结果如下：

v1: 12 11 10 30

v2: 12 11 10 30

v3: 10 10 10

v4: 10 20 30 40 0 0

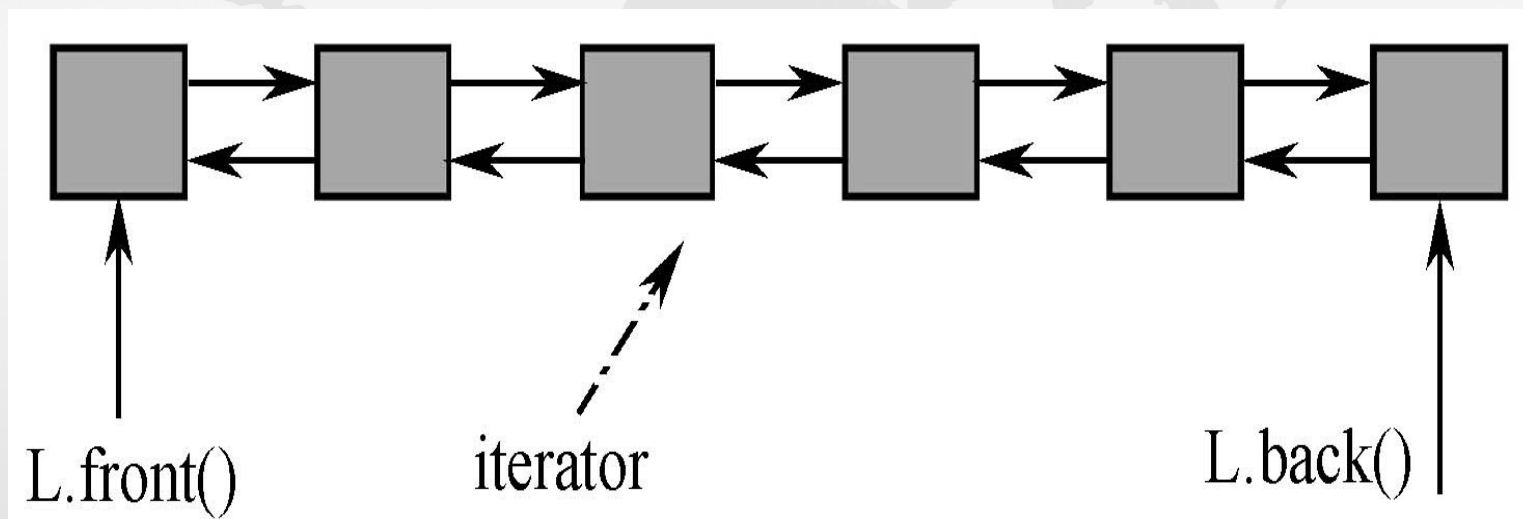
v4: 0 0 0 0 0 0 0 40 30 20

10

## 7.5.2 顺序容器

### 2. List

- STL 中的 `list` 是一个双向链表，可以从头到尾或从尾到头访问链表中的节点，节点可以是任意数据类型。链表中节点的访问常常通过迭代器进行。下图是一个链表的示意图。



# 链表的操作

## （ 1 ）链表的构造（模板参数 T 是链表的数据类型）

`list<T> c`          建立一个空链表 `c`

`list<T> c1(c2)`      建立与 `c2` 同型的链表 `c1` （ `c2` 的每个元素都被复制）

`list<T> c(n)` 建立具有 `n` 个元素的链表 `c`，元素值由默认构造函数产生

`list<T> c(n,e)`      建立 `n` 个元素的链表 `c`，每个元素的值都是 `e`

`list<T> c(beg, end)` 建立链表 `c`，并用 `[beg, end]` 区间内的元素作初始化

`c.~list<e>()`      销毁链表 `c`，释放内存

## （ 2 ）链表赋值

`c1=c2`              将 `c2` 链表的全部元素赋值给 `c1` 链表

`c1.assign(n,e)`      将元素 `e` 拷贝 `n` 次到 `c1` 链表

`c.assign(beg,end)` 将区间 `[beg,end]` 的元素赋值给 `c`

`c1.swap(c2)`        将链表 `c1` 和 `c2` 的全部元素互换



### ( 3 ) 链表存取

c.front()            返回第一个元素，不检查元素存在与否  
c.back()            返回最后一个元素，不检查元素存在与否

### ( 4 ) 链表插入与删除

c.insert(pos,e)        在 pos 位置插入元素 e 的副本，并返回新元素的位置  
c.insert(pos,n,e)       在 pos 位置插入元素 e 的 n 个副本，没有返回值  
c.insert(pos, beg, end)    在 pos 位置插入区间 [ beg, end] 内的全部元素  
c.push\_back(e)        在尾部追加一个元素 e 的副本  
c.push\_front(e)       在表头插入元素 e 的一个副本  
c.pop\_back(e)        删除最后一个元素  
c.pop\_front()        删除第一个元素  
c.remove(val)        删除值为 val 的元素  
c.remove\_if(op)       删除所有“造成 op(e) 结果为 true”的元素  
c.erase(pos)        删除 pos 指向的元素，返回下一元素的位置  
c.erase(beg, end)    删除区间 [beg,end] 内的元素，返回下一元素位置  
c.resize(n)        将链表 c 的大小重新设置为 n  
c.clear()        删除链表所有元素，将整个容器置空

# 链表的操作

## ( 5 ) 链表的特殊操作

|                                  |                                       |
|----------------------------------|---------------------------------------|
| c.unique()                       | 删除相邻重复元素，只留一个                         |
| c.unique(op)                     | 若存在若干相邻且使 op() 操作为 true 的元素，删除重复，只留一个 |
| c1.splice(pos, c2)               | 将 c2 内的所有元素转换到 c1 内，pos 之前            |
| c1.splice(pos, c2, c2pos)        | 将 c2 链表的 c2pos 所指元素移到 c1 内的 pos 指向的位置 |
| c1.splice(pos, c2, c2beg, c2end) | 将 c2 内 [c2beg, c2end] 区间的所有元素转换到 c1 内 |
| pos 之前                           |                                       |
| c.sort()                         | 以 operator< 为准则，对所有元素排序               |
| c.sort(op)                       | 以 op() 为准则，对所有元素排序                    |
| c1.merge(c2)                     | c2 合并到 c1，若合并前有序则合后仍有序                |
| c.reverse()                      | 将所有元素反序                               |

## 7.5.2 顺序容器

【例 7-21】 list 应用的一个例子。

```
//Eg7-21.cpp
#include<iostream>
#include<list> // 链表头文件
using namespace std;
void main(){
    int i;
    list<int> L1,L2;
    int a1[]={100,90,80,70,60};
    int a2[]={30,40,50,60,60,60,80};
```



## 7.5.2 顺序容器

```
for(i=0;i<5;i++)  
    L1.push_back(a1[i]);           // 将 a1 数组加入到 L1 链表中  
for(i=0;i<7;i++)  
    L2.push_back(a2[i]);           // 将 a2 数组加入到 L2 链表中  
L1.reverse();                     // 将 L1 链表倒序  
L1.merge(L2);                     // 将 L2 合并到 L1 链表中  
cout<<"L1 的元素个数为: "<<L1.size()<<endl;  
L1.unique();                       // 删除 L1 中相邻位置的相同元素, 只留 1 个  
while(!L1.empty()){  
    cout<<L1.front()<<"\t";  
    L1.pop_front();               // 删除 L1 的链首元素  
}  
cout<<endl;  
};
```

本程序的运行结果如下:

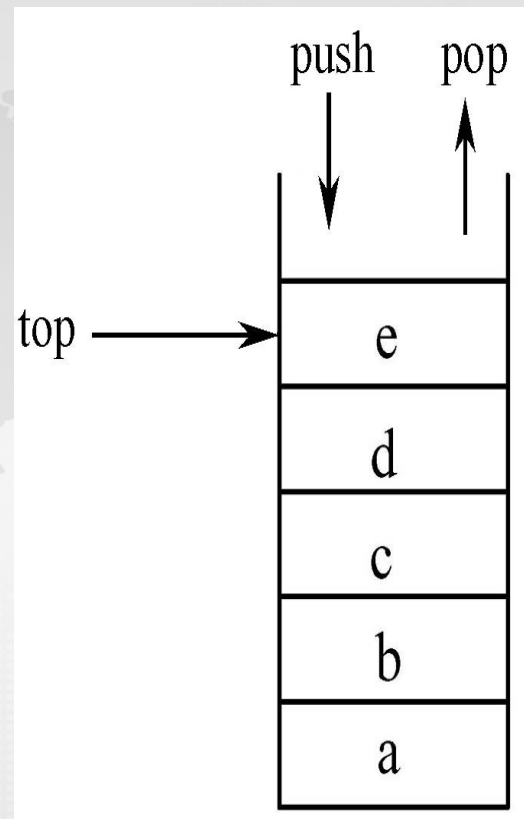
L1 的元素个数为: 12

30 40 50 60 70 80 90 100

## 7.5.2 顺序容器 - 适配器

### 3 . Stack

- 堆栈（**stack**）是一种较简单的常用容器，它是一种受限制的向量，只允许在向量的一端存取元素，后进栈的元素先出栈，即 **LIFO**（**last in first out**）。右图是一个字符堆栈的示意图。
- 可以用 **list**、**vector** 实现 **Stack**，因此是一种容器适配器。



## 7.5.2 顺序容器

- STL 中的堆栈提供的主要操作如下：
  - push() 将一个元素加入 **stack** 内，加入的元素放在栈顶
  - top() 返回栈顶元素元素的值
  - pop() 删除栈顶元素
- top 与 pop 是不同的，top 只返回栈顶元素的值，不删除元素，而 pop 只删除栈顶元素，不返回值。



## 7.5.2 顺序容器

### 【例 7-22】 STL stack 应用的例子。

```
//Eg7-22.cpp
#include<iostream>
#include<stack>
using namespace std;
void main(){
    stack<int> s;
    s.push(10); s.push(20); s.push(30);
    cout<<s.top()<<"\t";
    s.pop();      s.top()=100;
    s.push(50); s.push(60);
    s.pop();
    while(!s.empty()){
        cout<<s.top()<<"\t";
        s.pop();
    }
    cout<<endl;
}
```

程序运行结果（分析其由来）：

30 50 100 10

## 7.5.2 顺序容器

### 4 . String

- **string** 可以被看成是以字符（ **characters** ）为元素的一种容器，字符构成序列（字符串），有时需要在字符序列中进行遍历，标准 **string** 类提供了 **STL** 容器接口，具有成员函数 **begin()** 和 **end()**，迭代器可以用这两个函数进行定位。
- **STL** 中的 **string** 是一种特殊类型的容器，原因是它除了可作为字符类型的容器外，更多的是作为一种数据类型——字符串，可以像 **int**、**double** 之类的基本数据类型那样定义 **string** 类型的数据，并进行各种运算。

## 表 7-5 string 的重载运算符

| 运算符 | 举例（s1、s2 是 string 类型） | 说 明                       |
|-----|-----------------------|---------------------------|
| =   | s2=s1                 | 赋值运算，将 s1 赋值给 s2          |
| >   | s1>s2                 | 若 s1 大于 s2，结果为真，否则为假      |
| ==  | s1==s2                | 若 s1 等于 s2，结果为真，否则为假      |
| >=  | s1>=s2                | 若 s1 大于或等于 s2，结果为真，否则为假   |
| <   | s1<s2                 | 若 s1 小于 s2，结果为真，否则为假      |
| <=  | s1<=s2                | 若 s1 小于或等于 s2，结果为真，否则为假   |
| !=  | s1!=s2                | 若 s1 不等于 s2，结果为真，否则为假     |
| +=  | s1+=s2                | 将 s2 连接在 s1 后面，并赋值给 s1    |
| []  | s[1]='a'              | string 可用数组方式访问元素，起始下标为 0 |



# ( 1 ) string 的常用成员函数

在下面的 string 成员函数介绍中，假设 s1 、 s2 的定义如下：

```
string s1="ABCDEFGFH";  
string s2="0123456123";  
string s;
```

- ① **substr(n1, n)** 取子串函数，从当前字符串的 n1 下标开始，取出 n 个字符。如 “ s=s1.substr(2, 3)” 的结果为： s="CDE"
- ② **swap(s)** 交换字符串。如 “ s1.swap(s2)” 的结果为： s1="0123456123" ， s2="ABCDEFGFH"
- ③ **size()/length()** 计算字符串中当前存放的字符个数。如 “ s1.length()” 的结果为： 7
- ④ **capacity()** 计算字符串的容量（可容纳的字符个数）。如 “ s1.capacity()” 的结果为： 31
- ⑤ **max\_size()** 计算 string 类型数据的最大容量。如 “ s1.max\_size()” 的结果为： 4294967293
- ⑥ **find(s)** 在当前字符串中查找子串 s ， 如找到就返回 s 在当前串中的起始位置；若没有找到，返回常数 string::npos 。如 “ s1.find("EF")” 的结果为： 4
- ⑦ **rfind(s)** 同 find ， 但从后向前进行查找。如 “ s1.rfind("BCD")” 的结果为： 1

# ( 1 ) string 的常用成员函数

- ① **find\_first\_of(s)** 在当前串中查找子串 s 第一次出现的位置。如 “ s2.find\_first\_of("123") ” 的结果为：  
1
- ② **find\_last\_of(s)** 在当前串中查找子串 s 最后一次出现的位置。如 “ s2.find\_last\_of("123") ” 的结果为：  
9
- ③ **replace(n1, n, s)** 替换当前字符串中的字符， n1 是替换的起始下标， n 是要替换的字符个数， s 是用来替换的字符串。如 “ s1.replace(2, 3, s2) ” 的结果为： s1="AB0123456123FH"
- ④ **replace(n1, n, s, n2, m)** n1 是替换的起始下标， n 是替换掉的字符个数， s 是用来替换的字符串， n2 是 s 中用来替换的起始下标， m 是 s 中用于替换的字符个数。如 “ s1.replace(2, 3, s2, 2, 3) ” 的结果为： s1="AB234FH"
- ⑤ **insert(n, s)** 在当前串的下标位置 n 之前，插入 s 串。如 “ s1.insert(2, "88888") ” 的结果为：  
s1="AB88888CDEFH"
- ⑥ **insert(n1, n, s, n2, m)** 在当前串的 n1 下标后插入 s 串， n2 是 s 串中要插入的起始下标， m 是 s 串中要插入的字符个数。如 “ s1.insert(2, s2, 3, 2) ” 的结果为： s1="AB34CDEFH"





## 7.5.2 顺序容器

【例 7-23】 string 应用的例子。

//Eg7-23.cpp

```
#include<iostream>
```

```
#include<string>
```

```
using namespace std;
```

```
void main(){
```

```
    string s1=" 中华人民共和国成立了 ";
```

```
    string s2=" 中国人民从此站起来了! ";
```

```
    string s3,s4,s5;
```

```
    s3=s1+" , "+s2;
```

## 7.5.2 顺序容器

```
int n=s1.find_first_of(" 人民 ");
    if (n!=string::npos)
        cout<<" 人民在 s1 中的位置: "<<n<<endl;
    else      cout<<" 在 s1 中没有该子串! ";
    s4=s1.substr(4,10);
    cout<<"s1= "<<s1<<endl;
    cout<<"s2= "<<s2<<endl;
    cout<<"s3= "<<s3<<endl;
    cout<<"s4= "<<s4<<endl;
    if (s1>s2) cout<<"s1>s2= true "<<endl;
    else      cout<<"s1>s2= false"<<endl;
    s3.replace(s3.find(" 从此 "),4," 从 1949 年 ");
    cout<<"s3 after replace= "<<s3<<endl;
    s3.insert(s3.find(" 站 "),"10 月 ");
    cout<<"s3 after insert= "<<s3<<endl;
}
```

## 7.5.2 顺序容器

- 程序运行结果如下：

人民在 s1 中的位置： 4

s1= 中华人民共和国成立了

s2= 中国人民从此站起来了！

s3= 中华人民共和国成立了，中国人民从此站起来了！

s4= 人民共和国

s1>s2= true

s3 after replace= 中华人民共和国成立了，中国人民从 1949 年站起来了！

s3 after insert= 中华人民共和国成立了，中国人民从 1949 年 10 月站起来了！



不是 STL 容器的是 ( )

- ☐ A list
- ☐ B deque
- ☐ C stack
- ☒ D iterator

提交

## 7.5.3 迭代器

### 1、迭代器的概念

- 迭代器（**iterator**）是一个对象，常用它来遍历容器，即在容器中实现“取得下一个元素”的操作。
- 迭代器的操作类似于指针，但它是基于模板的“功能更强大、更智能、更安全的指针”，用于指示容器中的元素位置，通过迭代器能够遍历容器中的每个元素。
- 迭代器是 **STL** 的核心，定义了哪些算法在哪些容器可以使用，把算法和容器连接起来，使算法、容器和迭代器能够协同工作，实现强大的程序功能。
- 若某个容器要使用迭代器，就必须定义迭代器。定义迭代器时，必须指定迭代器所使用的容器类型。

## 7.5.3 迭代器

### 2、迭代器的基本操作

- ①在容器中的特定位置定位迭代器。
- ②在迭代器指示位置检查是否存在对象。
- ③获取存储在迭代器指示位置的对象值。
- ④改变迭代器指示位置的对象值。
- ⑤在迭代器指示位置插入新对象。
- ⑥将迭代器移到容器中的下一个位置。



## 7.5.3 迭代器

### 3、迭代器提供的主要操作

- `operator *` 返回当前位置上的元素值
- `operator++` 将迭代器前进到下一个元素位置
- `operator--` 将迭代器后退到前一个元素位置
- `operator==` 或 `operator!=`
- `operator=` 为迭代器赋值
- `begin()` 指向容器起点（即第一个元素）位置
- `end()` 指向容器的结束点
- `rbegin()` 指向按反向顺序的第一个元素位置之前
- `rend()` 指向按反向顺序的最后一个元素后的位置

## 7.5.3 迭代器

【例 7-24】 链表迭代器应用举例。

//Eg7-24.cpp

```
#include<iostream>
```

```
#include<list>
```

```
using namespace std;
```

```
int main(){
```

```
    int i;
```

```
    list<int> L1, L2, L3(10);
```

```
    list<int>::iterator iter;           // 定义迭代器 iter
```

```
    int a1[]={100,90,80,70,60};
```

```
    int a2[]={30,40,50,60,60,60,80};
```

```
    for(i=0;i<5;i++)
```

```
        L1.push_back(a1[i]);
```

```
        for(i=0;i<7;i+
```

```
    +)
```

```
        L2.push_front(a2[i]);
```

```
for(iter=L1.begin(); iter!=L1.end(); iter++)
    cout<<*iter<<"\t"
cout<<endl;
int sum=0;
// 通过迭代器反向输出 L2 的所有元素
for(iter=--L2.end();iter!=L2.begin();iter--){
    cout<<*iter<<"\t";
    sum+=*iter;    // 计算 L2 所有链表节点的总和
}
cout<<"\nL2: sum="<<sum<<endl;
int data=0;
// 通过迭代器修改 L3 链表的内容
for(iter=L3.begin();iter!=L3.end();iter++)
    *iter=data+=10;

for(iter=L3.begin();iter!=L3.end();iter++)
    cout<<*iter<<"\t";
cout<<endl;
return 0;
}
```

程序运行结果如下:

100 90 80 70 60

30 40 50 60 60 60

L2: sum=300

10 20 30 40 50 60 70 80 90 100



```

// Eg7-25.cpp
#include<iostream>
#include<list>
#include<string>
using namespace std;
struct Student {
    string name;
    int age;
    void outData() { cout<<name<<"\t"<<age<<"\t\t"; }
};
int main() {
    list<Student> L = {"张三", 21}, {"李四", 18}, {"黄明", 27};
    double avg = 0;
    int n = 0;
    for (auto iter = L.begin(); iter != L.end(); iter++) {
        iter->outData();
        avg += iter->age;
        n++;
    }
    cout<<"avg = "<<avg/n<<endl;
    n = 0; avg = 0;
    for(auto iter = begin(L); iter != end(L); iter++) {
        iter->outData();
        avg += iter->age;
        n++;
    }
    cout<<"avg = "<<avg /n<< endl;
    for(auto x:L)
        x.outData();
}

```

**【例 7-25】** 设计一个包括多名学生的链表，每位学生有姓名和年龄，输出每名学生的姓名和年龄，以及所有学生的平均年龄。

程序运行结果如下：

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 27 | 张三 | 21 | 李四 | 18 | 黄明 |
| 27 | 张三 | 21 | 李四 | 18 | 黄明 |
| 27 | 张三 | 21 | 李四 | 18 | 黄明 |
| 27 | 张三 | 21 | 李四 | 18 | 黄明 |

## 7.5.4 pair 和 tuple 容器

### 1、关于 pair 和 tuple 容器

- tuple 与 pair 都是一种容器模板，pair 只能有两个元素；
- tuple 称为元组，可以有任意多个不同类型的元素（但一个定义好的 tuple 类型的成员数目是固定的）。
- Tuple 可以用 STL 中的其它容器，如 list、vector、map、set 等作为元组的成员，建立随意而功能强大的数据结构。

# 7.5.4 pair 和 tuple 容器

## 2、tuple 对象的构造

- 定义一个 tuple 时，需要指出每个成员的类型。  

```
tuple<T1,T2.....,Tn> t; // 使用默认构造函数  
tuple<T1,T2,.....Tn> t(v1,v2.....vn); // 使用指定值初始化  
tuple<T1,T2,.....Tn> t{v1,v2.....vn}; // 用值列表初始化
```
- 例如，
- ```
tuple<int, string,char*> t1,t2{ 1," 数据结构 ","3.5 学分 "};
```
- ```
tuple<string, vector<double>, int, list<int>> someVal("constants", { 3.12,2.34,32 }, 42, { 0,1,1,1 });
```
- 注意：tuple 的构造函数是 explicit 的，可以像 t2 那样用列表直接初始化，但不能用列表赋值方式初始化，如下所示：
- ```
tuple<int,int,int > t={1,2,3}; // 错误 原因 tuple 禁用了 operator=
```
- ```
tuple<int,int,int > t{1,2,3}; // 正确
```



# 7.5.4 pair 和 tuple 容器

## 3、tuple 元素访问

- tuple 中的每个成员都是 public 属性，且都可以是对象、数组之类的复杂数据类型。C++ 标准模板中提供了一个 get 函数模板，它以类似于数组下标索引的方式访问元组中指定位置的成员，用法如下：

**get<i>(t)**    //t 是元组，i 是元组中的元素位置，第 1 个元素位置为 0

- 例如
- ```
tuple<char *, int, vector<string>, list<int>>>
tue("tom", 101, { " 语文 "," 数学 "," 英语 " }, { 76,87,91 });
```
- ```
get<0>(tue)           // 返回考生姓名:      “ tom”
```
- ```
get<1>(tue)           // 返回考生考号:      101
```
- ```
get<2>(tue)           // 返回考试科目:      { " 语文 "," 数学 "," 英语 " }
```
- ```
get<3>(tue)           // 返回科目成绩:      { 76,87,91 }
```
- ```
get<3>(tue)[0]        // 返回 { 76,87,91 } 中的第一个元素，即 76
```

# 7.5.4 pair 和 tuple 容器

## 4、tuple 操作

- 同类型的 tuple 可以进行 <、==、= 运算。
- 用 make\_tuple 函数构造 tuple 对象，或者调用参数的类型推断 tuple 类型等。
- 例如，

```
auto t=make_tuple("string",3,2.1);
```

编译器会据实参推断元组类型 t 为： tuple<const char\*,int,double> ， 等价于下面的定义：

```
tuple<const char *,int,double> t("string", 3, 2.1);
```

```
struct A { string name;int score; };  
tuple < string, int, string, vector<A> >  
s1{"tom", 101, "信息管理", { {"math", 90}, {"eng", 78} } };
```

获取 tom 同学 math 成绩的方法是

- ☐ A << get<3>(s1)[0].score;
- ☐ B << get<2>(s1).score;
- ☐ C << get<3>(s1)[1].score;
- ☐ D << get<2>(s1)[0].score;

提交



## 7.5.4 pair 和 tuple 容器

### 5、tuple 应用

- 当希望将一些类型不同但具有联系的数据组合成单一对象，而又不想定义类或结构时，用元组把这些数据组合起来（快而随意的数据结构）就显得非常有用。
- 用元组作为函数的返回类型，可以一次返回多个数据。

**【例 7-26】** 在一个成绩系统中，要求函数返回的成绩数据包括：学生姓名，专业，班主任，大学英语，数学，C 语言等 5 科成绩，用元组处理数据可以简化程序设计。

- 下面的程序用来说明元组的解决本应用的方法。其中，inputData 函数用于说明为元组中的成员输入数据，并通过函数返回元组的方法；display 函数用于说明向函数传递元组参数的方法。

## 7.5.4 pair 和 tuple 容器

//Eg7-26.cpp

```
#include <tuple>
```

```
#include<string>
```

```
#include<list>
```

```
#include <vector>
```

```
#include<iostream>
```

```
using namespace std;
```

```
struct Grade {
```

// 表示学生科目、成绩的数据结构

```
    string courseName;
```

```
    double grade;
```

```
    Grade(string s, double g) :courseName(s), grade(g) {}
```

```
};
```

```
typedef tuple<string, int, string, vector<Grade>> Student;
```

`inputData` 函数返回的 `Student` 是一种复杂的元组数据类型，该元组保存学生的姓名，学号，专业，以及任意多门课程的成绩。

```
typedef tuple<string, int, string, vector<Grade>> Student;
```

```
Student inputData() {  
    Student stu;  
    cout << " 输入学生数据：姓名，学号，专业 " << endl;  
    cin >> get<0>(stu) >> get<1>(stu) >> get<2>(stu);    // 元组元素访问方法  
    string cName;  
    double score=0;  
    int i = 1;  
    while (cName != "exit") {  
        cout << " 输入第  " << i++ << " 科目名称，输入 excit 结束 :\t";  
        cin >> cName;  
        if (cName == "exit") break;  
        cout << " 成绩 :\t";  
        cin >> score;  
        get<3>(stu).push_back(Grade(cName, score));    // 向元组向量添加对象  
    }  
    return stu;  
}
```



**//typedef tuple<string, int, string, vector<Grade>> Student;**

```
void display(Student student) {  
    cout << get<0>(student) << "\t" << get<1>(student) << "\t"  
        << get<2>(student) << "\t" << endl;  
    //for 循环示范了访问元组中具有不确定个数的向量元素的访问方法。  
    for (int i = 0; i < get<3>(student).size(); i++)  
        cout << get<3>(student)[i].courseName << "\t"  
            << get<3>(student)[i].grade << endl;  
}  
void main(){  
    auto t = make_tuple("string", 3, 20.01);  
    tuple<char *,int,double> tt("string", 3, 20.01);  
    //tuple<int, int, int > t = { 1,2,3 };           // 错误  
    tuple<int, int, int > t5{ 1,2,3 };               // 正确  
    tuple<int, string,char*> t1,t2{ 1," 数据结构 ","3.5 学分 " };  
    t1 = t2;                                         // 同类型元组可以赋值  
    cout << get<0>(t1) << "\t" << get<1>(t1) << "\t"  
        << get<2>(t1) << endl;                     // 元组访问的常规方法
```

// 下面的代码段示例了具用链表、向量元素的元组定义方法，以及元组中的链表访问方法

```
tuple<string,vector<double>,int,list<int>>vtable("tomoto",{3.12,2.34}, 42, {10,8,1});  
list<int>::iterator iter;           // 访问链表的迭代器  
for (iter = get<3>(vtable).begin(); iter != get<3>(vtable).end(); iter++)  
    cout << *iter << "\t";  
    cout << endl;
```

**//student** 对象示例了向元组中的对象数组赋值的方法。

```
Student student{" 李四 ",1011," 计算机专业 ",{{" 英语 ",76.4},{ " 高数 ",87},{ "C++ 语言 ",89}}};  
display(student);  
student = inputData();  
cout << endl;  
display(student);  
}
```

# 程序运行结果

```
C:\Windows\system32\cmd.exe

1      数据结构      3.5学分
10     8      1      9
李四   1011   计算机专业
英语   76.4
高数   87
C++语言 89
输入学生数据: 姓名, 学号, 专业
张大明 1201 计算机软件
输入第 1 科目名称, 输入成绩:      数据库原理
Excit结束:
成绩:      89
输入第 2 科目名称, 输入成绩:      数据结构
Excit结束:
成绩:      76
输入第 3 科目名称, 输入成绩:      C++程序语言
Excit结束:
成绩:      97
输入第 4 科目名称, 输入成绩:      exit
Excit结束:

张大明 1201 计算机软件
数据库原理 89
数据结构 76
C++程序语言 97
请按任意键继续. . .
```



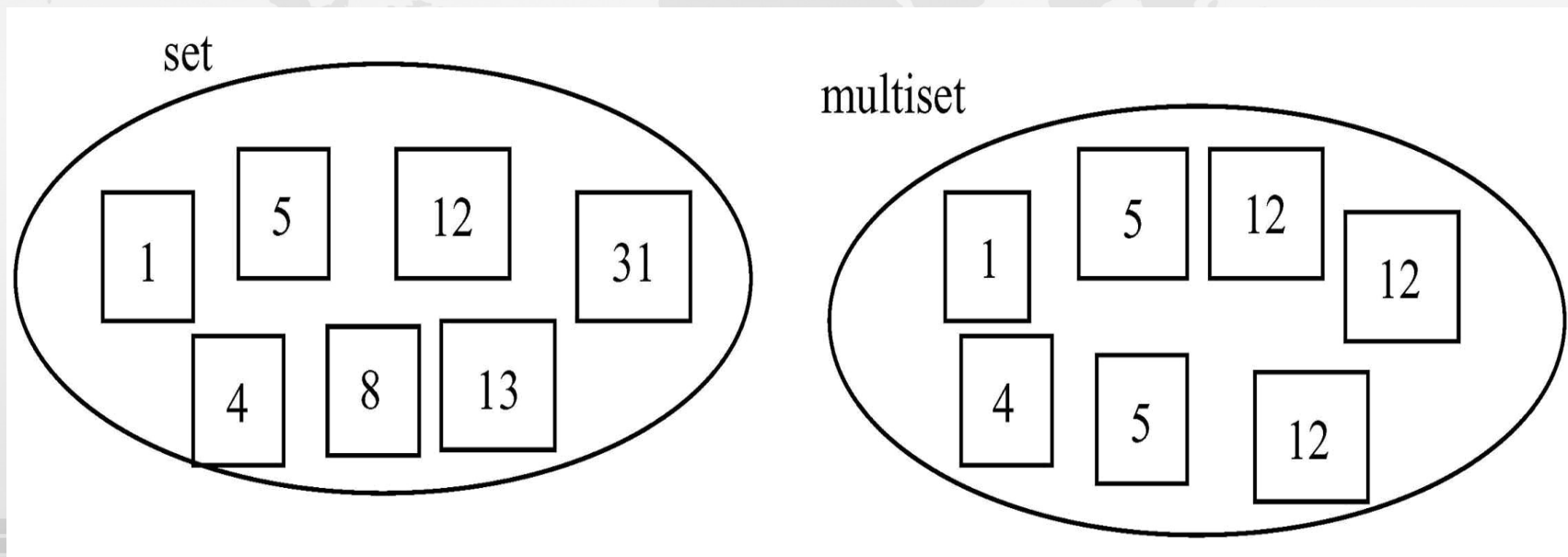
## 7.5.5 关联式容器

- 关联式容器类型
  - 集合（ `set` 、 `multiset` ）
  - 映射（ `map` 、 `multimap` ）
- 容器元素
  - 容器中的每个元素都有一个键和值， `set` 的键就是值， `map` 中的键值和实值分开，形成一种映射，因此 `map` 也称为映射表或字典。
- 实现技术
  - 容器内部结构是平衡二叉树（主要是红黑树）或 `hash-table` 。在每种关联容器中，关键字按顺序排列，容器遍历就以此顺序进行。并按关键字进行元素存储和查找。
  - 关联式容器没有头尾之说，只有最大、最小元素。因此没 `push_back()` 、 `push_front()` 、 `pop_back()` 、 `pop_front()` 这样的操作。

## 7.5.5 关联式容器

### 1. set 和 multiset

- **multiset** 和 **set** 提供了控制数字（包括字符及串）集合的操作，集合中的数字称为关键字，不需与另一个值与关键字相关联。
- **set** 和 **multiset** 会根据特定的排序准则，自动将元素排序，两者提供的操作方法基本相同，只是 **multiset** 允许元素重复而 **set** 不允许重复。



## 7.5.5 关联式容器

### ( 1 ) set 和 multiset 的定义

- `set c` 建立一个空的 `set/multiset` 集合
- `set c(op)` 以 `op` 为排序准则建立一个空集
- `set c1(c2)` 建立一个集合 `c1`，并用 `c2` 集合初始化
- `set c(beg, end)` 用区间 `[beg, end]` 建立一个集合 `c`
- `set` 可以是：
- `set/multiset<T>` 建立 `T` 类型的，以 `less<>`（从小到大）的排序集合
- `set/multiset<T, op>` 建立 `T` 类型的，以 `op` 指定排序规则的集合
  - 其中，`op` 可以是 `less<>` 或 `greater<>` 之一，应用时须在 `<>` 中写上类型，如 `greater<int>`。
    - `less` 指定排序方式为从小到大
    - `greater` 指定排序方式为从大到小，默认排序方式为 `less`。



## 7.5.5 关联式容器

### ( 2 ) set 和 multiset 的赋值比较运算

- set 和 multiset 支持  $>$ 、 $>=$ 、 $<$ 、 $<=$ 、 $!=$ 、 $==$  比较运算。例如，若有集合  $c1$ 、 $c2$ ，可以用  $c1==c2$ ， $c1>c2$  对它们进行相等或大于判断。
- 可以赋值运算符 “=” 进行集合赋值，如  $c1=c2$ 。

### ( 3 ) set 和 multiset 计算容量

size()            计算容器的大小

empty()        判断容器是否为空，若为空则返回 0

max\_size()      返回容器能够保存的最大元素个数

## 7.5.5 关联式容器

### ( 4 ) set 和 multiset 常用操作

|                  |                                      |
|------------------|--------------------------------------|
| count(e)         | 计算集合中元素 <b>e</b> 的个数                 |
| find(e)          | 查找集合中第 1 次出现元素 <b>e</b> 的位置          |
| lower_bound(e)   | 查找集合中第 1 个 “元素值 $\geq e$ ” 的位置       |
| upper_bound(e)   | 查找集合中第 1 个 “元素值 $> e$ ” 的位置          |
| insert(e)        | 在当前集合中插入元素 <b>e</b> ;                |
| insert(pos, e)   | 将 <b>e</b> 插入到 <b>pos</b> 位置         |
| insert(beg, end) | 将 <b>[beg, end]</b> 区间内的所有元素插入到当前集合中 |
| erase(e)         | 删除集合中的元素 <b>e</b>                    |
| erase(pos)       | 删除集合中指定位置 <b>pos</b> 的元素             |
| erase(beg,end)   | 删除区间 <b>[beg,end]</b> 的所有元素          |
| clear()          | 清空集合                                 |
| begin()          | 指向第 1 个元素位置，常与迭代器结合应用                |
| end()            | 指向最后元素的下一位置，常与迭代器结合应用                |

## 7.5.5 关联式容器

【例 7-27】 集合应用的例子。

```
//Eg7-27.cpp
#include<iostream>
#include<string>
#include<set>
using namespace std;
void main(){
    int a1[]={-2,0,30,12,6,7,12,10,9,10};
    set<int,greater<int> >set1(a1,a1+7);          set<int,greater<int>
    >::iterator p1;          set1.insert(12); set1.insert(12); // 向集合插入元素
    set1.insert(4);
    for(p1=set1.begin();p1!=set1.end();p1++)
        cout<<*p1<<" "; // 输出集合中的内容，它是从大到小的
    cout<<endl;
    string a2[]={" 杜明 "," 王为 "," 张清山 "," 李大海 "," 黄明浩 ",
        " 刘一 "," 张三 "," 林浦海 "," 王小二 "," 张清山 "};
```



```
// 定义字符串的 multiset 集合，默认排序从小到大
multiset<string>set2(a2,a2+10);
    multiset<string>::iterator p2;
    set2.insert(" 杜明 "); set2.insert(" 李则 ");
    for(p2=set2.begin();p2!=set2.end();p2++)
        cout<<*p2<<" ";           // 输出集合内容
    cout<<endl;
    string sname;
    cout<<" 输入要查找的姓名: ";
    cin>>sname;                       // 输入要在集合中查找的姓名
    p2=set2.begin();
    bool s=false;                     //s 用于判定找到姓名与否
    while(p2!=set2.end()){
        if(sname==*p2) {             // 如果找到就输出姓名
            cout<<*p2<<endl;
            s=true;
        }
        p2++;
    }
    if(!s)
        cout<<sname<<" 不在集合中! "<<endl; // 如果没有找到就给出提示
}
```

## 7.5.5 关联式容器

【例 7-28】 设计包括多名学生的集合对比 set 和 multiset 的区别，每位学生有姓名和年龄，输出各位学生的姓名和年龄，以及所有学生的平均年龄。

**设计思路：**设计 Student 结构存取和输出学生姓名和年龄信息。由于集合（set/multiset）中的元素是有序的，因此**必须定义 Student 对象大小比较的方法**，可以通过重载运算符函数 Student::operator<() 来实现。

由于 set 模板的默认比较函数 operator<() 是 const 函数，所以 Student 的**重载运算符函数也必须设置为 const 函数**。

【例 7-28】 设计包括多名学生的集合对比 **set** 和 **multiset** 的区别，每位学生有姓名和年龄，输出各位学生的姓名和年龄，以及所有学生的平均年龄。

```
// Eg7-28.cpp
#include<iostream>
#include<set>
#include<string>
using namespace std;
```

```
struct Student {
    string name;
    int age;
    bool operator<(const Student& o) const { return age < o.age; } // L1
    void outData()const { cout<<name<<"\t"<<age<<"\t\t"; }
};
```

```
int main() {
    set<Student> s1 = {"张三",21}, {"李四",18}, {"黄明",27}; //
L2
    multiset<Student> s2 = {"张三",21}, {"李四",18}, {"黄明",27}; //
L3
    s1.insert(Student({"张三", 21})); // L4
    s2.insert(Student({"张三", 21})); // L5
    for(auto iter = s1.begin(); iter != s1.end(); iter++) {

        iter->outData();

    }
    cout<<endl;
    for (auto iter = s2.begin(); iter != s2.end(); iter++) {
```

程序运行结果如下：

李四, 18  
李四, 18  
黄明, 27

张三, 21  
张三, 21

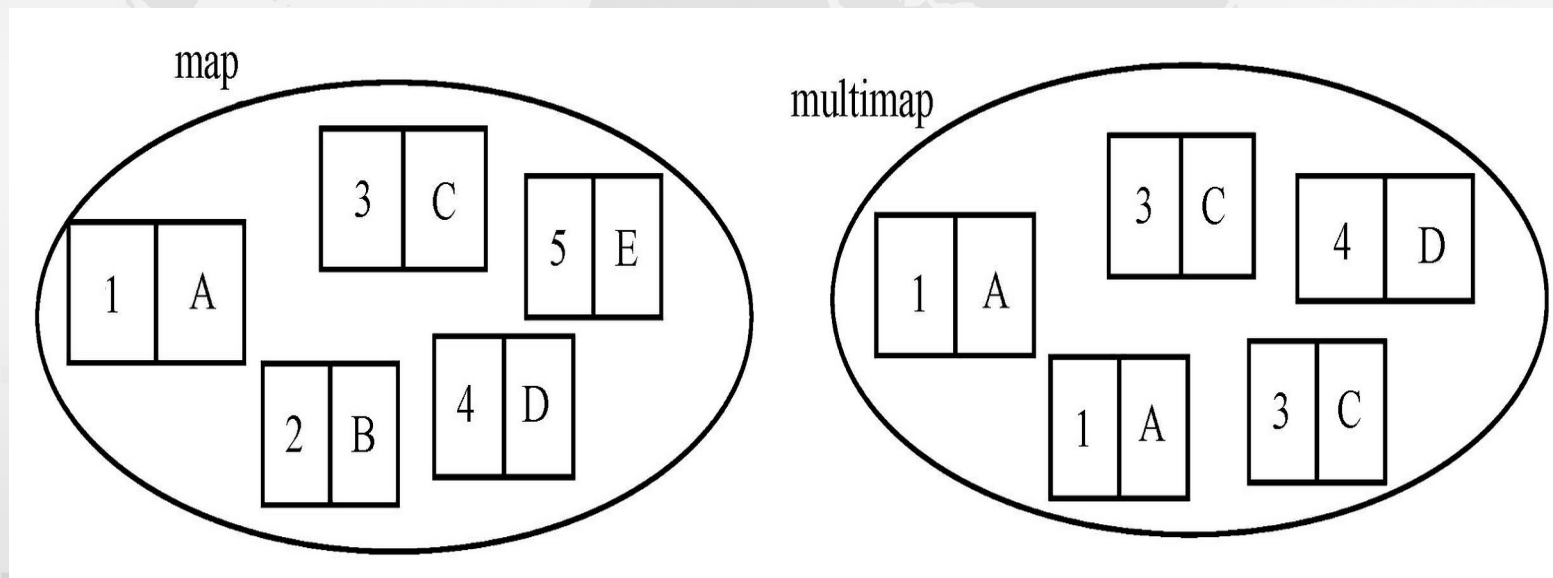
黄明, 27  
张三, 21



## 7.5.5 关联式容器

### 2. map 和 multimap

- **map** 和 **multimap** 提供了操作 < 键, 值 > 对的方法（其中的值也称为映射值），它们存储一对对象，即键对象和值对象，键对象是用于查找过程中的键，值是与键对应的附加数据
- **map** 中的元素不允许重复，而 **multimap** 中的元素是可以重复的。



## 7.5.5 关联式容器

### ( 1 ) map 和 multimap 的常用操作

- ① `insert(e)`      // 将元素 `e` 插入到 `map` , `multimap` , `set` , `multiset`
- ② `make_pair(e1,e2)`
- ③ `map/multimap` 类型的迭代器提供了以下两个数据成员：
  - `first` , 用于访问键;
  - `second` , 用于访问值

## 7.5.5 关联式容器

【例 7-29】 用 multimap 构造汉英对照字典。

```
//Eg7-29.cpp
#include<iostream>
#include<string>
#include<map>
using namespace std;
void main() {
    multimap<string, string> dict;           // dict 是用于存放字典的
    multimap
    multimap<string, string>::iterator p; // 定义访问字典的迭代器
    string eng[] = {"cliff", "berg", "precipice", "tract"};
    string che[] = {"悬崖", "冰山", "悬崖", "一片, 区域"};
    for(int i = 0; i < 4; i++)
        dict.insert(make_pair(eng[i], che[i])); // 批量插入字典元素
```



// 插入单个元素

```
dict.insert(make_pair(string("tract"), string(" 地带 ")));
dict.insert(make_pair(string("precipice"), string(" 危险的处境 ")));
dict.insert(make_pair("day", " 一天 "));           // L1 , 正确
// dict["precipice"] = " 悬崖, 峭壁 ";           // L2 , 错误
for(p = dict.begin(); p != dict.end(); p++) // 输出字典内容
    cout<<p->first<<"\t" <<p->second<<endl; // first 是键, second 是值
string word;
cout<<" 请输入要查找的英文单词: ";
cin>>word;
for(p = dict.begin(); p != dict.end(); p++) // 遍历字典, 查找单词
    if(p->first == word)
        cout<<p->second<<endl; // 输出单词的中文解释
cout<<" 请输入要查找的中文单词: ";
cin>>word;
for(p = dict.begin(); p != dict.end(); p++) // 遍历字典, 查找汉语
    if(p->second == word)
        cout<<p->first<<endl; // 输出汉语对应的英语单词
}
```

程序运行结果如下：

|           |        |
|-----------|--------|
| berg      | 冰山     |
| cliff     | 悬崖     |
| precipice | 悬崖     |
| precipice | 危险的处境  |
| tract     | 一片, 区域 |
| tract     | 地带     |

precipice

悬崖  
危险的处境

: 悬崖

cliff  
precipice

## 7.5.5 关联式容器

【例 7-30】 用 map 查询学生的专业，学生包括学号和姓名。

设计思路：

- 设计结构或类 Student 存储学生信息，设计 map 映射学生和他的专业，用迭代器遍历 map 来查找学生的专业。
- 由于 Student 对象作为 map 的 < 键，值 > 中的键，因此还需要设计比较 Student 对象大小的函数，这里通过普通函数重载 operator<( ) 实现。

```
#include<iostream>
#include<map>
#include<string>
using namespace std;
class Student {
    string name;
    int age;
public:
    Student(string Name, int Age):name(Name), age(Age) {}
    void outData()const{ std::cout<<name<<","<<age<<"\t"; }
    string getName()const { return name; }
    int getAge()const { return age; }
};
```

【例 7-30】 用 map 查询学生的专业，学生包括学号和姓名。

```
bool operator<(Student a, Student b) {
    return a.getAge()<b.getAge(); }

int main() {
    map<Student, string> m = {{Student(" 张三 ", 21), " 信管专业 "},
    {Student(" 黄明 ", 27), " 会计专业 " } };
    m[Student( " 王五 ", 22)] = " 会计学 ";
    m.insert(make_pair(Student(" 张三 ", 24), " 经济学 "));
    for (auto iter = m.begin(); iter != m.end(); iter++) {
        (iter->first).outData();
        cout<<iter->second<<endl;
    }
    string name;
    bool find = false;
    cout<<endl<<" 输入想查询的学生姓名: ";
    cin>>name;
    for (auto iter = m.begin(); iter != m.end(); iter++) {
        if ((iter->first).getName() == name) {
            (iter->first).outData();
            cout<<iter->second<<endl;
            find = true;
        }
    }
    if(!find)
        cout<<" 没有找到该学生! "<< endl;
    return 0;
}
```

程序运行结果如下：

|        |      |
|--------|------|
| 张三， 21 | 信管专业 |
| 王五， 22 | 会计学  |
| 张三， 24 | 经济学  |
| 黄明， 27 | 会计专业 |

： 张三

|        |      |
|--------|------|
| 张三， 21 | 信管专业 |
| 张三， 24 | 经济学  |



# 统计英文文章中的单词及出现次数

## 解题思路：

利用 **map< 键, 值 >** 的数据构造特点，用单词作键，出现次数作值，直接将文章中的单词读到键中，同时增加值。

```
#include <iostream>
#include<vector>
#include<map>
#include<string>
#include<fstream>
using namespace std;
```

```
vector<string> ReadDataFromFileText(string textfile)
```

```
{
```

```
    ifstream fin(textfile);
```

```
    string strWord("");
```

```
    vector<string> v;
```

```
    while (fin >> strWord)
```

```
    {
```

```
        // 字符串 string 类的比较要用 compare 函数
```

```
        if ((strWord.compare(strWord.length() - 1, 1, ".") == 0)
```

```
            || (strWord.compare(strWord.length() - 1, 1, ",") == 0))
```

```
        {
```

```
            strWord.erase(strWord.length() - 1, 1);
```

```
        }
```

```
        v.push_back(strWord);
```

```
    }
```

```
    return v;
```

```
}
```

本函数与题意无关，此处仅是演示 **vector** 的用途和从文中读取单词的方法

```
map<string,int> wordCount(string file)
{
    ifstream fin(file);
    string strWord("");
    map <string, int>wcount;
    while (fin >> strWord)
    {
        // 字符串 string 类的比较要用 compare 函数
        if ((strWord.compare(strWord.length() - 1, 1, ".") == 0)
            || (strWord.compare(strWord.length() - 1, 1, ",") == 0))
        {
            strWord.erase(strWord.length() - 1, 1);
        }
        wcount[strWord]++;
    }
    return wcount;
}
```



```
void main()
{
    map<string, int>word;
    vector<string> v;
    v=ReadDataFromFileText("d:\\abc.txt");
    for (auto x : v) {
        cout << x<<" ";
    }
    word = wordCount("d:\\abc.txt");
    for (auto x : word) {
        cout << x.first << "\\t" << x.second << endl;
    }
}
```

下面的容器中，没有 pop\_back() 操作的是

- ☐ A list
- ☐ B vector
- ☒ C set\map 和 stack
- ☐ D deque

提交

## 7.5.6 算法

- 算法（ **algorithm** ）是用模板技术实现的适用于各种容器的通用程序。算法常常通过迭代器间接地操作容器元素，而且通常会返回迭代器作为算法运算的结果。
- **STL** 大约提供了 **100** 个算法，每个算法都是一个模板函数或者一组模板函数，能够在许多不同类型的容器上进行操作，各个容器则可能包含着不同类型的数据元素。**STL** 中的算法覆盖了在容器上实施的各种常见操作，如遍历、排序、检索、插入及删除元素等操作



# 7.5.6 算法

## 1. find 和 count 算法

- **find** 用于查找指定数据在某个区间中是否存在，该函数返回等于指定值的第一个元素位置，如果没有找到就返回最后元素位置； **count** 用于统计某个值在指定区间出现的次数，
- 其用法如下：
  - `find(beg,end,value)`
  - `count(beg,end,value)`
  - **[beg, end]** 是指定的区间，常用迭代器位置描述该区间， **value** 是要查找或统计的值。

# 7.5.6 算法

## 【例 7-31】 find 算法举例。

```
//Eg7-31.cpp
#include<iostream>
#include<list>
#include<algorithm>
using namespace std;
void main(){
    int arr[]={100,200,300,400,500,500,600,700,800,900,1000};
    int *ptr;
    ptr=find(arr,arr+9,400);           // 查找 400 在 arr 数组中的地址
    cout<<"400 在数组中的下标是: "
        <<ptr-arr<<endl;              //find 返回地址,
    list<int> L1;                       // 定义链表 L1
    int a1[]={30,40,50,60,60,60,80};
    for(int i=0;i<7;i++)
        L1.push_back(a1[i]);           // 将 a1 数组加入到 L1 链表中
```

## 7.5.6 算法

```
list<int>::iterator pos;  
pos=find(L1.begin(),L1.end(),80);           // 在 L1 中查找 80 ， 结果放于 pos 中  
if(pos!=L1.end())  
    cout<<"L1 链表中存在数据元素: "<<*pos;           // 输出找到的数据  
    cout<<" ， 它是链表中的第: "<<distance(L1.begin(),pos)+1  
        <<" 个节点! "<<endl;    // 计算迭代器与链首元素间隔的元素个数  
int n1=count(arr,arr+10,500);    // 统计 arr 数组中 500 的个数  
int n2=count(L1.begin(),L1.end(),60);    // 统计 L1 链表中 60 的个数  
cout<<"arr 数组中有: "<<n1<<" 个 "<<500<<endl;  
cout<<"L1 链表中有: "<<n2<<" 个 "<<60<<endl;  
}
```

程序运行结果如下:

400 在数组中的下标是: 3

L1 链表中存在数据元素: 80 ， 它是链表中的第: 7 个节点!

arr 数组中有: 2 个 500

L1 链表中有: 3 个 60



# 7.5.6 算法

## 2. merge

`merge` 可对两容器进行合并，将结果存放在第 3 个容器中，

- 其用法如下：

`merge(beg1, end1, beg2, end2, dest)`

- `merge` 将 `[beg1, end1]` 与 `[beg2, end2]` 区间合并，把结果存放在 `dest` 容器中。如果参与合并的两个容器中的元素是有序的，则合并的结果也是有序的。
- 说明： `list` 链表也提供了一个 `merge` 成员函数，它能够把两个 `list` 类型的链表合并在一起。同样，如果合并前的链表是有序的，则合并后的链表仍然有序。

## 【例 7-32】 merge 算法与 list 的 merge 成员函数的应用。

```
#include<iostream>
#include<list>
#include<algorithm>
using namespace std;
void main(){
    int a1[]={10,20,30,40,50,60,70};
    int a2[]={40,50,60};
    int a[10];
    merge(a1,a1+7,a2,a2+3,a);        // 将 a1 、 a2 合并，结果放在 a 数组中
    for(int i=0;i<10;i++)    cout<<a[i]<<"\t";
    cout<<endl;
    list<int> L1,L2;
    list<int>::iterator pos;        //pos 迭代器用于输出链表元素
    for(int i=0;i<7;i++)
        L1.push_back(a1[i]);        // 插入 L1 的链表元素
    for(int j=0;j<3;j++)
        L2.push_back(a2[j]);        // 插入 L2 的链表元素
    L1.merge(L2);                    // 用 list 的 merge 成员合并 L1 、 L2
    for(pos=L1.begin();pos!=L1.end();pos++) // 用迭代器输出合并后的 L1
        cout<<*pos<<"\t";
    cout<<endl;
}
```

# 7.5.6 算法

## 3. search 算法

- search 算法则是从一个容器查找由另一个容器所指定的顺序值。
- search 用法形式如下：

`search(beg1,end1,beg2,end2)`

search 将在 [beg1, end1] 区间内查找有无与 [beg2, end2] 相同的子区间，如果找到就返回 [beg1, end1] 内第一个相同元素的位置，如果没找到，返回 end1; search 将在 [beg1, end1] 区间内查找有无与 [beg2, end2] 相同的子区间，如果找到就返回 [beg1, end1] 内第一个相同元素的位置，如果没找到，返回 end1;



## 7.5.6 算法

【例】 search 算法举例：查找某数据区间在另一数据区间中是否存在

```
#include<iostream>
#include<vector>
#include<list>
#include<algorithm>
using namespace std;
void main(){
    int a1[]={10,20,30,40,50,60,70,80};
    int a2[]={40,50,60};
    int *ptr;
    ptr=search(a1,a1+8,a2,a2+3);    // 查找 a2 数组在 a1 中的位置
    if(ptr==a1+8)
        cout<<"no match found\n";
    else
        cout<<"a2 match a1 at:"<<(ptr-a1)<<endl; // 输出第一个匹配元素的位置
```

## 7.5.6 算法

```
vector<int> v;  
list<int> L;  
for(int i=0;i<8;i++)  
    v.push_back(a1[i]);    // 将 a1 数组插入 v 向量  
for(int j=0;j<3;j++)  
    L.push_back(a2[j]);    // 将 a2 数组插入 L 链表  
vector<int>::iterator pos;  
pos=search(v.begin(),v.end(),L.begin(),L.end());  
    // 在 v 中查找 L  
cout<<distance(v.begin(),pos)<<endl;  
    //distance 计算找到元素在 V 中的下标  
}
```

## 7.5.6 算法

### 4 . sort

`sort` 可对指定容器区间内的元素进行排序，默认的排序方式是从小到

- 其用法如下：

`sort(beg,end)`

`[beg, end]` 是要排序的区间，`sort` 将按从小到大的顺序对该区间的元素进行排序。



### 【例 7-33】 利用 sort 算法对数组和向量进行排序。

```
//Eg7-33.cpp
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
void main(){
    int a1[]={10,-20,30,4,50,13,7};
    int a2[]={-2,0,30,12,6,7,-9,56,32,78};
    sort(a1,a1+7);           // 排序 a1 数组
    cout<<"a1[]: ";
    for(int i=0;i<7;i++)
        cout<<a1[i]<<"\t";
    cout<<endl;
    vector<int> L1;
    vector<int>::iterator pos;
    for(i=0;i<10;i++)
        L1.push_back(a2[i]); // 将 a2 数组插入到 L1 链表中
    sort(L1.begin(),L1.end()); // 排序 L1
    cout<<"L1: ";
    for(pos=L1.begin();pos!=L1.end();pos++)
        cout<<*pos<<"\t"; // 输出 L1 链表中的值
    cout<<endl;
}
```

## 7.5.7 STL 容器和算法处理自定义类的常见问题

### ■ 代码无问题却报出大量错误信息

- 初用 STL 库处理自定义类对象时，可以会遇到编译器一下报出了上百个错误。遇到这样的问题时不要慌，**可能就一两个错误**。
- 可以**先排除程序本身的语法错误**，再根据具体的错误信息检查是否是模板参数不匹配，或者**某项内容与 const 参数不符合**。
- 最常见的错误是应用 STL 中的 **set**、**map**、**sort**、**min**、**max** 等容器和算法处理自定义类对象时，没有重载**小于或大于运算符函数**，无法进行自定义类对象排序所产生的错误。原因是 **set** 和 **map** 容器是有序的（包括 **multiset** 和 **multimap** 等），在将元素插入这类容器时就要进行对象的大小比较，如果类对象没有提供大小比较函数就无法排序，程序就会产生错误。
- 如果自定义类没有定义大小比较方法，在用 **sort**、**min**、**max** 等算法处理这样的类对象时也会产生错误。解决的方法是**为类提供比较大小的函数**，最简单的方法是重载小于比较运算符函数，形式如下：

```
class A {  
    .....  
    // set、map 要求存入其中的对象的比较函数为 const 函数  
    bool operator<(const A& o) const {...}  
    .....  
};
```

【例 7-34】 定义简单的复数类，建立该复数的数组和 set 集合，并用 STL 中的 sort 算法对数组进行排序输出。

```
// Eg7-34.cpp
#include<iostream>
#include<set>
#include<algorithm>
using namespace std;

class Complex {
private:
    double i, r;
public:
    Complex(double ir, double rr):i(ir), r(rr) {}
    void set_r(double pr) { r = pr; }
    void set_i(double pi) { i = pi; }
    const double& real()const { return r; }
    double& image() { return i; }
    // bool operator<(const Complex &o)const { return r < o.r; } // L1
    void outdata() {
        if (i >= 0)
            cout<<r<<"+"<<i<<"i"<<endl;
        else
            cout<<r<<i<<"i"<<endl;
    }
};
```

```
int main() {
    Complex c1[] = {{1.0,1.0}, {6.0,2.0}, {3.0,3.0}};
    cout<<"-----sort array output-----"<<endl;
    sort(c1, c1+2);
    for(Complex c:c1) c.outdata();
    cout<<"-----sort set output-----"<<endl;
    set<Complex>cset;
    for(Complex c:c1)
        cset.insert(c);
    Complex c2{4.0, 4.0}, c3{1.0, 1.0};
    cset.insert(c2); //L2
    cset.insert(c3); //L3
    for(Complex c:cset)
        c.outdata();
}
```

程序将产生编译错误，取消 L1 位置的注释符，程序就能够正常运行！原因是 L2、L3 会默认调用小于比较运算符函数



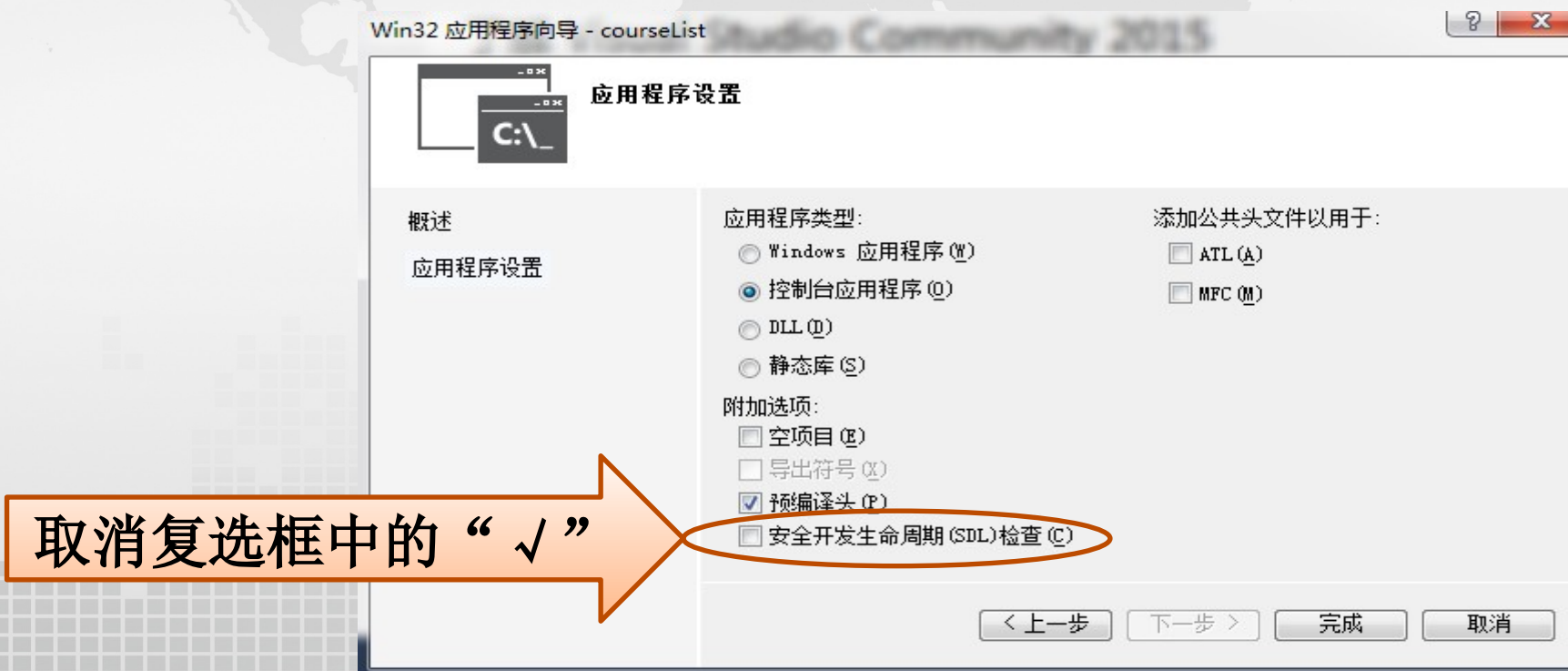
## 7.6 编程实作：模板和 STL 编程应用



## 7.6 编程实作：模板和 STL 编程应用

### 1. 建立项目并编写主程序

- 启动 Visual C++2022，选择“文件 | 新建 | 项目”菜单命令，在 C 盘根目录下建立一个新项目，项目名为 courseList。在建立项目时，**取消“安全开发生命周期（SDL）检查”**
- 进入项目的程序编辑窗口，修改该项目的源文件“courleList.cpp”，下面是其全部代码：

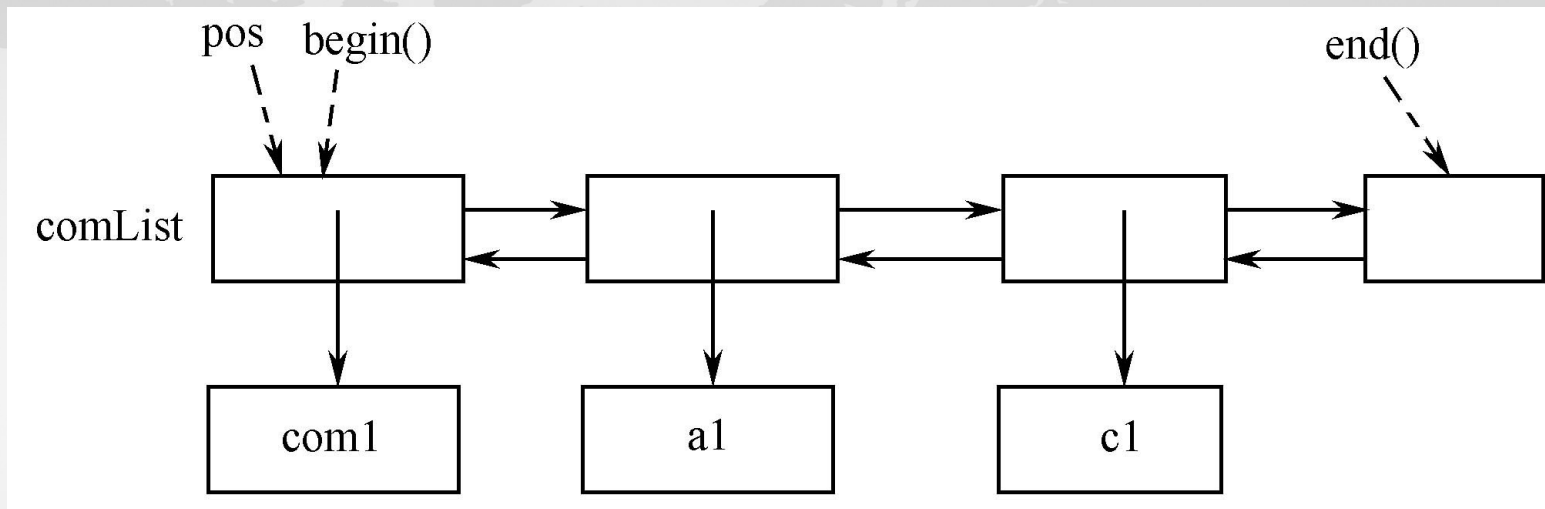


```
// courselist.cpp : 定义控制台应用程序的入口点。  
//
```

```
#include "stdafx.h"  
#include<iostream>  
#include<list>  
#include"Chemistry.h"  
#include"Account.h"  
using namespace std;  
void main() {  
    list<comFinal*> comList;// 定义基类 comFinla 对象的指针链表  
    list<comFinal*>::iterator pos;  
    comFinal com1(" 阿曼 ", 76, 87, 90);  
    Account a1(" 张三星 ", 98, 90, 97, 90, 90);  
    Chemistry c1(" 光红顺 ", 89, 80, 80, 80, 80);  
    comList.push_back(&com1);// 将基类 comFinal 对象的指针加入链表  
    comList.push_back(&a1);// 将派生类 Account 的对象指针加入链表  
    comList.push_back(&c1);// 将派生类 Chemitry 的对象指针加入链表  
    for (pos = comList.begin(); pos != comList.end(); pos++)  
        (*pos)->show();// 遍历链表, 输出各对象的数据成员  
}
```



## 7.6 编程实作：模板和 STL 编程应用



## 7.6 编程实作：模板和 STL 编程应用

### 2. 在项目中加入各个类的程序代码

- 将第 6 章建立的目录 C:\course 下的 comFinal.h、Account.h、Chemistry.h 以及 comFinal.cpp、Account.cpp、Chemistry.cpp 复制到 courseList.cpp 所在的目录中。
- 选择菜单“工程 | 添加工程 | files”，将上述各类的头文件和源码文件添加到当前工程中。

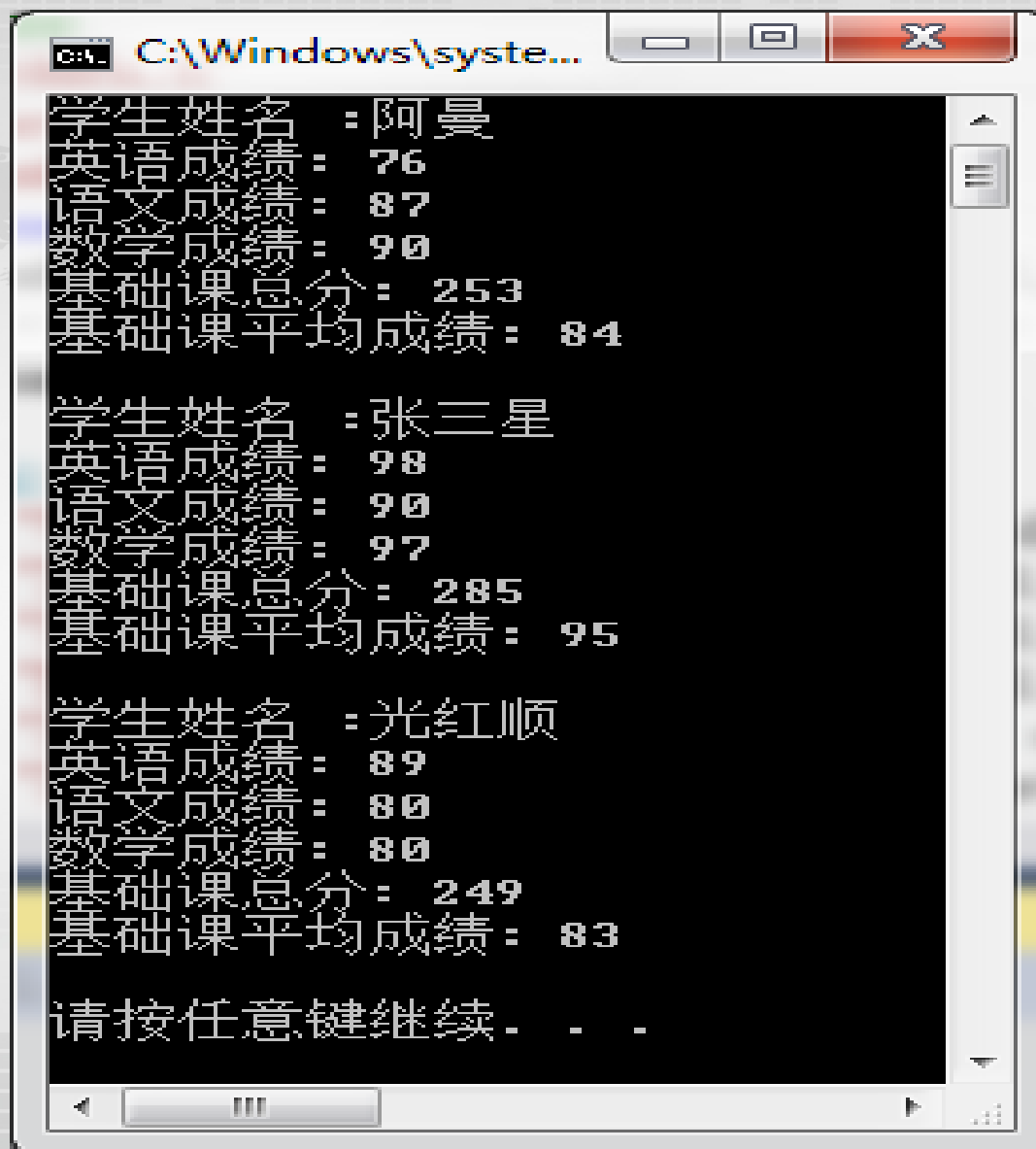
### 3. 验证程序

- 编译运行 courseList 程序。
- 若在编译时出现类似下面的错误
- 错误 C1010 在查找预编译头时遇到意外的文件结尾。是否忘记了向源中添加“#include "stdafx.h"”? courseListc:\courselist\courselist\comfinal.cpp 17
- 则在 comFinal.cpp、Account.cpp、Chemistry.cpp 三个源码文件的开头添加下代的代码行：

```
#include "stdafx.h"
```

## 7.6 编程实作：模板和 STL 编程应用

- 程序运行结果



```
C:\Windows\system...
学生姓名 : 阿曼
英语成绩 : 76
语文成绩 : 87
数学成绩 : 90
基础课总分 : 253
基础课平均成绩 : 84

学生姓名 : 张三星
英语成绩 : 98
语文成绩 : 90
数学成绩 : 97
基础课总分 : 285
基础课平均成绩 : 95

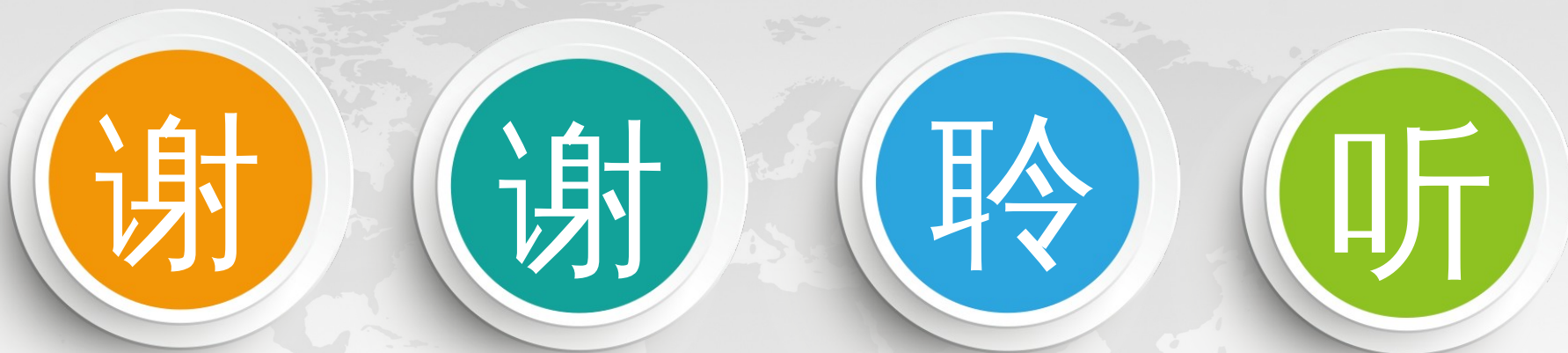
学生姓名 : 光红顺
英语成绩 : 89
语文成绩 : 80
数学成绩 : 80
基础课总分 : 249
基础课平均成绩 : 83

请按任意键继续 . . .
```



# 小结： STL 与 operator< 运算符

- STL 容器大多具有排序功能，这些容器在默认情况下，用 operator< 运算符对容器中的对象进行升序排序。
- 容器中的排序算法在默认情况下也用 operator< 运算符对容器中的对象进行大小比较，按升序排序。
- 如  
list set/multiset map/multiset sort
- 因此，必须对 operator< 进行重载实现自定义对象的大小比较，才能够将对象插入容器。
- 通用方法：以友元重载 operator<
- Class A{
  - bool operator<(const A&right);
  - friend bool operator<(const A &left,const A &right)
  - }



THANKS!